Quick Reference Tables — All Templates

The full Quick Reference Table for every template category, in one place. Columns: **# · Name · Description · Intuition · Time · Space · Key Implementation**. See each category's own file for the worked solutions.

Two Pointers `two_pointers.md` (91 problems)

#	Name	Description	Intuition	Time	Space	Key Implementation
167	Two Sum II	Sorted array. Find two numbers summing to target. Return 1-based indices.	Squeeze from both ends; each comparison tells you which side is wrong, so one move eliminates a whole row/column of pairs.	O(n)	O(1)	Standard
15	3Sum	Find all unique triplets summing to 0. No duplicate triplets in output.	Pin one number, then it becomes Two Sum on the rest — turning a triple search into a sweep.	O(n^2)	O(1) excluding output (O(n) sort stack)	Standard
18	4Sum	Find all unique quadruplets summing to target.	Pin two numbers, then the inner search is Two Sum — same idea as 3Sum with one more fixed level.	O(n^3)	O(1) excluding output (O(n) sort stack)	Standard
11	Container With Most Water	Given heights of vertical lines, find two lines forming the container with most water.	Width shrinks every step, so only a taller boundary can ever help; abandon the shorter wall.	O(n)	O(1)	Standard
42	Trapping Rain Water	Given elevation heights, compute total water trapped after rain.	Water level at a cell is set by the shorter side's tallest wall, so always process from the lower side where that max is already known.	O(n)	O(1)	Standard
27	Remove Element	Remove all occurrences of `val` in-place. Return new length.	The writer only advances on keepers, so it always points at the next free slot for a kept value.	O(n)	O(1)	Standard
26	Remove Duplicates from Sorted Array	Remove duplicates in-place so each element appears at most once. Return new length.	Since the array is sorted, comparing against the last written value is enough to drop every duplicate run.	O(n)	O(1)	Standard
80	Remove Duplicates from Sorted Array II	Each element may appear at most twice in-place. Return new length.	Looking two slots back lets at most two copies through before the third is rejected.	O(n)	O(1)	Standard
283	Move Zeroes	Move all zeroes to end while preserving relative order of non-zero elements.	Compact the non-zeros to the front in order, then pad the leftover tail with zeros.	O(n)	O(1)	Standard
75	Sort Colors	Sort array of 0s, 1s, 2s in-place without library sort.	A single scan grows a "small" region from the left and a "large" region from the right, leaving mids in the middle.	O(n)	O(1)	Standard
1	Two Sum	Given an integer array and a target sum, return the indices of two numbers that add up to target. Exactly one solution exists.	Trade space for time — remember every value you've passed so the complement is a single lookup.	O(n)	O(n)	Standard
16	3Sum Closest	Find three integers whose sum is closest to target. Return that sum.	Same pin-and-sweep as 3Sum, but instead of matching exactly you keep the running closest.	O(n^2)	O(1)	Standard
31	Next Permutation	Rearrange the array to its lexicographically next greater permutation in-place. If none, sort ascending.	The longest decreasing suffix is already maximal; bump the digit just before it up to the smallest larger value, then reset the suffix to ascending.	O(n)	O(1)	Standard
41	First Missing Positive	Find the smallest missing positive integer in an unsorted array. O(n) time, O(1) space.	Use the array itself as a hash table keyed by value, then scan for the first slot holding the wrong number.	O(n)	O(1)	Standard
48	Rotate Image	Rotate an n×n matrix 90 degrees clockwise in-place.	Transposing flips across the main diagonal; reversing each row then completes the clockwise turn.	O(n^2)	O(1)	Standard
54	Spiral Matrix	Return all elements of an m×n matrix in spiral order.	Peel the matrix like an onion, one boundary layer per loop.	O(m*n)	O(1) excluding output	Standard
73	Set Matrix Zeroes	If any element is 0, set its entire row and column to 0, in-place.	Reuse the first row and column as the bookkeeping space so no extra arrays are needed.	O(m*n)	O(1)	Standard
128	Longest Consecutive Sequence	Find the length of the longest run of consecutive integers in an unsorted array. O(n).	Each sequence is counted exactly once by starting only at its smallest member.	O(n)	O(n)	Standard
164	Maximum Gap	Find the maximum gap between successive elements in the sorted form of an array. O(n).	With buckets sized at the minimum possible gap, the answer never lies inside a bucket, only between bucket boundaries.	O(n)	O(n)	Standard
169	Majority Element	Find the element appearing more than n/2 times.	Pairing off different elements cancels them out; the majority element always survives.	O(n)	O(1)	Standard
189	Rotate Array	Rotate the array right by k positions, in-place.	Two partial reversals after a full reversal place each block in its rotated position without extra space.	O(n)	O(1)	Standard
217	Contains Duplicate	Return true if any value appears at least twice.	A set rejects repeats, so the first failed insert proves a duplicate.	O(n)	O(n)	Standard
220	Contains Duplicate III	Are there indices i, j with `|nums[i]-nums[j]| <= valueDiff` and `|i-j| <= indexDiff` ?	Same-bucket means automatically within range; neighbor buckets need an explicit check. Slide a window of size indexDiff.	O(n)	O(min(n, indexDiff))	Standard

#	Name	Description	Intuition	Time	Space	Key Implementation
229	Majority Element II	Find all elements appearing more than n/3 times.	At most two elements can each occur more than a third of the time, so track two running candidates.	O(n)	O(1)	Standard
238	Product of Array Except Self	Return output where `output[i]` is the product of all elements except `nums[i]`. No division.	Each position's answer is everything to its left times everything to its right.	O(n)	O(1) excluding output	Standard
259	3Sum Smaller	Count triplets with sum strictly less than target in a sorted array.	Once the largest partner works, every smaller partner does too, so count them in bulk.	O(n^2)	O(1)	bulk-count all valid j for this i
280	Wiggle Sort	Reorder in-place so `nums[0] <= nums[1] >= nums[2] <= nums[3]...`	A greedy local swap is always safe — it never breaks the pair you already fixed.	O(n)	O(1)	Standard
289	Game of Life	Simulate one Game of Life step, updating all cells simultaneously, in-place.	Pack the new state into a spare bit so neighbor counts still read the old state during the pass.	O(m*n)	O(1)	Standard
303	Range Sum Query - Immutable	Answer many range-sum queries on a static array efficiently.	A range sum is the difference of two prefix sums, so each query is O(1) after one build pass.	O(n) build, O(1) query	O(n)	Standard
304	Range Sum Query 2D - Immutable	Answer many 2D region-sum queries on a static matrix efficiently.	Each region sum combines four corner prefix sums via inclusion-exclusion.	O(m*n) build, O(1) query	O(m*n)	Standard
307	Range Sum Query - Mutable	Support range-sum queries and point updates.	A Fenwick tree stores partial sums over power-of-two ranges so updates and queries each touch only log n nodes.	O(log n) update/query	O(n)	Standard
311	Sparse Matrix Multiplication	Multiply two sparse matrices, skipping zero elements.	A zero in A contributes nothing, so skip it entirely instead of doing a full triple loop.	O(m*k*n) worst case	O(m*n) excluding output	skip zero to exploit sparsity
315	Count of Smaller Numbers After Self	For each element, count elements to its right that are smaller. O(n log n).	During a merge, every time a right element is taken before a left one, it is a smaller-to-the-right for that left element.	O(n log n)	O(n)	Standard
325	Maximum Size Subarray Sum Equals k	Find the longest subarray with sum equal to k. O(n).	A subarray sums to k exactly when two prefix sums differ by k; keep the earliest index to maximize length.	O(n)	O(n)	Standard
327	Count of Range Sum	Count range sums that lie in [lower, upper]. O(n log n).	A range sum is a difference of two prefix sums; merge sort lets you count qualifying pairs across halves efficiently.	O(n log n)	O(n)	Standard
334	Increasing Triplet Subsequence	Return true if there exists `i < j < k` with `nums[i] < nums[j] < nums[k]`.	Maintaining the two smallest ascending values seen so far is enough to detect a third larger one.	O(n)	O(1)	Standard
349	Intersection of Two Arrays	Return the intersection of two arrays (unique elements only).	Set membership turns intersection into linear lookups.	O(n + m)	O(n)	Standard
350	Intersection of Two Arrays II	Return the intersection including duplicates (multiplicity is the min of counts).	A frequency map lets each shared occurrence be matched at most once.	O(n + m)	O(min(n, m))	Standard
370	Range Addition	Apply range updates `[start, end] += inc` then return the final array.	Mark only the endpoints of each update; a single prefix pass materializes all increments.	O(n + updates)	O(n)	Standard
384	Shuffle an Array	Return a uniformly random permutation of the array; support reset to original.	Choosing each position's element uniformly from the remaining ones yields every permutation with equal probability.	O(n) per shuffle	O(n)	Standard
414	Third Maximum Number	Return the third distinct maximum; if it doesn't exist, return the maximum.	Keep a sorted top-three as you scan, skipping duplicates.	O(n)	O(1)	Standard
419	Battleships in a Board	Count battleships ('X' runs, horizontal or vertical, never adjacent).	Each ship has exactly one cell with no ship-neighbor above or to its left, so count those.	O(m*n)	O(1)	Standard
442	Find All Duplicates in an Array	Values in [1,n], each appears once or twice. Return the ones appearing twice. O(n) time, O(1) extra space.	Flip the sign at each value's home index; encountering an already-negative slot means that value repeated.	O(n)	O(1)	Standard
448	Find All Numbers Disappeared in an Array	Values in [1,n]. Return all numbers in [1,n] missing from the array.	Use sign marking; any home index never marked means that number was absent.	O(n)	O(1)	Standard
454	4Sum II	Count quadruplets (i,j,k,l) with `A[i]+B[j]+C[k]+D[l] == 0`.	Split four arrays into two pairs so a hash of one pair's sums turns the search into O(n^2) lookups.	O(n^2)	O(n^2)	Standard
457	Circular Array Loop	Detect a cycle of length > 1 with consistent direction in a circular array of jumps.	Fast/slow pointers detect cycles; reject single-element self-loops and direction flips.	O(n)	O(1)	Standard
485	Max Consecutive Ones	Find the maximum number of consecutive 1s in a binary array.	Grow a streak while seeing 1s and reset at each 0.	O(n)	O(1)	Standard

#	Name	Description	Intuition	Time	Space	Key Implementation
493	Reverse Pairs	Count pairs (i,j) with `i < j` and `nums[i] > 2 * nums[j]`.	Sorted halves let you count `nums[i] > 2*nums[j]` pairs with a linear two-pointer sweep per merge.	O(n log n)	O(n)	Standard
498	Diagonal Traverse	Traverse an m×n matrix diagonally, alternating direction each diagonal.	Cells on the same diagonal share `r+c`; flip the read direction on alternating diagonals to zig-zag.	O(m*n)	O(1) excluding output	Standard
517	Super Washing Machines	Minimize the max number of moves to equalize dresses across machines (one dress to a neighbor per move).	A bottleneck is either a machine with too much surplus or a cut that must pass a large net flow.	O(n)	O(1)	Standard
523	Continuous Subarray Sum	Is there a subarray of length >= 2 whose sum is a multiple of k?	Two prefix sums with the same remainder mod k bound a subarray sum divisible by k.	O(n)	O(min(n, k))	Standard
525	Contiguous Array	Find the longest subarray with equal numbers of 0s and 1s.	Equal counts means a running balance returns to a value seen before.	O(n)	O(n)	Standard
532	K-diff Pairs in an Array	Count unique pairs (i,j) where `nums[i] - nums[j] == k` (k >= 0).	Counting distinct values avoids duplicate pairs; the zero-diff case needs a repeated value.	O(n)	O(n)	Standard
560	Subarray Sum Equals K	Count subarrays summing to exactly k.	Each earlier prefix equal to `s-k` marks the start of a qualifying subarray ending here.	O(n)	O(n)	Standard
566	Reshape the Matrix	Reshape an m×n matrix into r×c keeping row-major order; return original if sizes mismatch.	Both shapes share a linear index, so map between them with division and modulo.	O(m*n)	O(r*c) excluding output	Standard
581	Shortest Unsorted Continuous Subarray	Find the shortest subarray that, if sorted, makes the whole array sorted.	The window starts where an element exceeds some later minimum and ends where an element falls below some earlier maximum.	O(n)	O(1)	Standard
594	Longest Harmonious Subsequence	Find the longest subsequence where max and min differ by exactly 1.	A harmonious subsequence uses exactly two adjacent values, so combine each pair's counts.	O(n)	O(n)	Standard
599	Minimum Index Sum of Two Lists	Find common strings between two lists with minimum index sum.	Index sum is minimized greedily as you scan; ties are collected together.	O(n + m)	O(n)	Standard
611	Valid Triangle Number	Count triplets from an array of side lengths that form a valid triangle.	Only the two smaller sides need to beat the largest, so fix the largest and bulk-count valid pairs.	O(n^2)	O(1)	bulk-count valid i for this j
661	Image Smoother	Replace each cell with the floor of the average of itself and its up-to-8 neighbors.	Average over the in-bounds 3×3 window centered on each cell.	O(m*n)	O(m*n) excluding output	Standard
723	Candy Crush	Repeatedly crush horizontal/vertical runs of 3+ same-positive candies and apply gravity, until stable.	Mark all matches in one pass before clearing so simultaneous crushes are handled, then let candies fall.	O((m*n)^2) worst case	O(1)	Standard
724	Find Pivot Index	Find the leftmost index where the sum to its left equals the sum to its right.	Track a running left sum and derive the right sum from the precomputed total.	O(n)	O(1)	Standard
766	Toeplitz Matrix	Check if every top-left to bottom-right diagonal has a single value.	Comparing each cell to its diagonal predecessor verifies all diagonals locally.	O(m*n)	O(1)	Standard
769	Max Chunks To Make Sorted	Array is a permutation of [0,n-1]. Max number of chunks that can be sorted independently to sort the whole.	When the largest value seen so far equals the index, everything up to here is a self-contained block.	O(n)	O(1)	Standard
795	Number of Subarrays with Bounded Maximum	Count subarrays whose maximum lies in [left, right].	"Max in range" equals the difference of two "max at most X" counts, each computed in one pass.	O(n)	O(1)	Standard
807	Max Increase to Keep City Skyline	Increase building heights as much as possible without changing skylines viewed from any of 4 sides.	A building is capped by the smaller of its row and column maxima to preserve both silhouettes.	O(n^2)	O(n)	Standard
825	Friends Of Appropriate Ages	Count friend requests; x friends y unless `age[y] <= 0.5*age[x]+7`, `age[y] > age[x]`, or `age[y]>100 && age[x] <100`.	Ages are bounded, so counting per age-bucket pair collapses the problem to constant-size work.	O(n + 120^2)	O(1)	Standard
838	Push Dominoes	Simulate falling dominoes given a string of 'L', 'R', '.'.	Each cell's outcome is the balance of forces from the nearest pushers on either side.	O(n)	O(n)	Standard
845	Longest Mountain in Array	Find the length of the longest mountain (strictly up then strictly down).	Scan to a peak by rising, then descend; the span counts only when both directions occurred.	O(n)	O(1)	Standard
896	Monotonic Array	Return true if the array is entirely non-increasing or entirely non-decreasing.	A single pass noting direction changes detects non-monotonicity.	O(n)	O(1)	Standard
912	Sort an Array	Sort an integer array. Implement an O(n log n) sort.	Recursively split, sort halves, and merge — stable O(n log n) without library sort.	O(n log n)	O(n)	Standard
974	Subarray Sums Divisible by K	Count subarrays whose sum is divisible by k.	Two prefixes with the same remainder mod k bound a divisible subarray; count pairs per remainder.	O(n)	O(k)	Standard

#	Name	Description	Intuition	Time	Space	Key Implementation
977	Squares of a Sorted Array	Return the squares of a sorted array in sorted order.	The biggest squares sit at the extremes, so fill the result from the largest slot inward.	O(n)	O(1) excluding output	Standard
1013	Partition Array Into Three Parts With Equal Sum	Can the array be split into three contiguous parts with equal sums?	Greedily close off a part each time the running sum hits one-third; succeed if at least three parts form.	O(n)	O(1)	Standard
1074	Number of Submatrices That Sum to Target	Count submatrices summing to target.	Reduce 2D to 1D by fixing the top and bottom rows, then count target-sum subarrays.	O(m^2 * n)	O(n)	Standard
1109	Corporate Flight Bookings	Given bookings `[first, last, seats]`, return total seats booked per flight.	Range increments become two endpoint marks resolved by one prefix pass.	O(n + bookings)	O(n)	Standard
1213	Intersection of Three Sorted Arrays	Return the common elements of three sorted arrays.	Like merging — advance whichever pointer lags so all three meet at shared values.	O(n)	O(1) excluding output	Standard
1275	Find Winner on a Tic Tac Toe Game	Given alternating A/B moves, determine the winner, "Draw", or "Pending".	Signed counts per line reveal a win the moment any line reaches +3 or -3.	O(1)	O(1)	Standard
1351	Count Negative Numbers in a Sorted Matrix	Count negatives in a matrix sorted descending by row and column.	From the bottom-left, sorted order lets each step rule out a whole row or column of negatives.	O(m + n)	O(1)	Standard
1424	Diagonal Traverse II	Traverse a jagged 2D list diagonally from bottom-left to top-right.	Cells on a diagonal share `r+c`; emitting rows in descending order gives the bottom-left-up direction.	O(total)	O(total)	Standard
1460	Make Two Arrays Equal by Reversing Sub-arrays	Can target become arr after reversing any sub-arrays any number of times?	Any permutation is reachable through reversals, so only element counts matter.	O(n)	O(1)	Standard
1480	Running Sum of 1d Array	Return the running (prefix) sum of the array.	Each running sum is the prior running sum plus the current value.	O(n)	O(1)	Standard
1498	Number of Subsequences That Satisfy the Given Sum Condition	Count subsequences where min + max <= target. Answer mod 1e9+7.	After sorting, fixing the minimum lets every subset of the elements up to the max be chosen freely.	O(n log n)	O(n)	Standard
1748	Sum of Unique Elements	Sum all elements that appear exactly once.	Count occurrences, then sum only the singletons.	O(n)	O(1)	Standard
1762	Buildings With an Ocean View	Return indices of buildings with nothing taller to their right (ocean to the right).	Sweeping from the ocean side, only record buildings that exceed every taller one already passed.	O(n)	O(1) excluding output	Standard
1868	Product of Two Run-Length Encoded Arrays	Multiply two run-length-encoded arrays element-wise; return the RLE product.	March through both run lists together, consuming the shorter overlap and coalescing equal results.	O(n + m)	O(1) excluding output	Standard
1877	Minimize Maximum Pair Sum in Array	Pair up elements to minimize the maximum pair sum.	Pairing extremes balances the sums, keeping the largest pair as small as possible.	O(n log n)	O(1)	Standard
1893	Check if All the Integers in a Range Are Covered	Are all integers in [left, right] covered by at least one given interval?	Mark interval starts and ends, then a prefix sweep shows which points are covered.	O(n + 50)	O(1)	Standard
1894	Find the Student that Will Replace the Chalk	Students consume chalk in order cyclically; find who runs out.	Whole cycles repeat, so only the remainder after a full round determines the failing student.	O(n)	O(1)	Standard
1920	Build Array from Permutation	Return `result[i] = nums[nums[i]]`.	Each output is a double lookup into the permutation.	O(n)	O(n) excluding output	Standard
1929	Concatenation of Array	Return `nums` concatenated with itself.	Copy each element into position `i` and `i+n`.	O(n)	O(1) excluding output	Standard

Sliding Window `sliding_window.md` (22 problems)

#	Name	Description	Intuition	Time	Space	Key Implementation
643	Maximum Average Subarray I	Find the contiguous subarray of length k with the maximum average value.	Max average of fixed length = max sum; slide a width-k window and track the best running sum.	O(n)	O(1)	Standard
567	Permutation in String	Return true if any permutation of s1 appears as a substring of s2.	A permutation is just a fixed-length window whose letter counts match s1, so slide and check the counts.	O(n)	O(1)	Standard
219	Contains Duplicate II	Return true if any two equal values are at most k indices apart.	Keep only the last k values in a set; a failed insert means a duplicate within k indices.	O(n)	O(k)	Standard
239	Sliding Window Maximum	Return the maximum of every contiguous subarray of size k.	A smaller value with a larger one still in the window can never be the max again, so discard it.	O(n)	O(k)	Standard
1004	Max Consecutive Ones III	Flip at most k zeros. Return the length of the longest subarray of 1s.	"Flip at most k zeros" = longest window containing at most k zeros; grow, and shrink only when zeros exceed k.	O(n)	O(1)	Standard
3	Longest Substring Without Repeating Characters	Find the length of the longest substring with all unique characters.	A repeat appears the moment you add it, so shrink from the left until that one character is unique again.	O(n)	O(1)	Standard
424	Longest Repeating Character Replacement	Replace at most k characters. Find the longest substring with one repeated char.	A window is valid if the non-dominant chars (size − maxFreq) fit within k replacements; otherwise shrink.	O(n)	O(1)	Standard
209	Minimum Size Subarray Sum	Find the minimum length subarray with sum ≥ target.	Once the window reaches target, every extra left-trim that stays ≥ target gives a shorter candidate.	O(n)	O(1)	Standard
76	Minimum Window Substring	Find the shortest substring of s that contains all characters of t.	Expand until all of t is covered, then shrink as far as you can while still covering it to find the tightest window.	O(n)	O(1)	Standard
438	Find All Anagrams in a String	Given strings `s` and `p`, return a list of all start indices of `p`'s anagrams in `s`. (An anagram uses same characters with same frequencies.)	An anagram is a fixed-length window whose letter counts equal p's, so slide width- `p.length()` and compare counts.	O(n)	O(1)	fixed window size
159	Longest Substring with At Most 2 Distinct Characters	Return the length of the longest substring with at most 2 distinct characters.	Grow the window freely; whenever a third distinct character appears, drop from the left until only 2 remain.	O(n)	O(1)	shrink when >2 distinct
340	Longest Substring with At Most K Distinct Characters	Return the length of the longest substring with at most `k` distinct characters.	Same as #159 but the distinct cap is `k`; shrink whenever the map holds more than k distinct characters.	O(n)	O(k)	Parameterized generalization of #159 — replace the hard-coded `2` with `k`.
1343	Number of Sub-arrays of Size K and Average Greater than or Equal to Threshold	Return the number of contiguous subarrays of size `k` whose average is greater than or equal to `threshold`.	"Average ≥ threshold" over fixed width k is just "sum ≥ k·threshold"; slide a width-k window and count the hits.	O(n)	O(1)	fixed-size sliding window. To avoid floating-point division, compare `windowSum >= k * threshold`.
30	Substring with Concatenation of All Words	Given `s` and an array `words` (all of equal length), return all start indices of substrings in `s` that are a concatenation of every word in `words` exactly once, in any order.	Every candidate substring has fixed length `words.length * wordLen`. Slide a word-aligned window: start at each offset `0..wordLen-1` and move in steps of `wordLen`, maintaining a frequency map of words seen.	O(n * wordLen)	O(words.length * wordLen)	word-aligned window, step = wordLen
187	Repeated DNA Sequences	Return all 10-letter substrings that occur more than once in a DNA string `s` (characters `A`, `C`, `G`, `T`).	Fixed window of size 10; record each substring in a set and report it the first time a duplicate insert fails.	O(n)	O(n)	fixed window size 10
395	Longest Substring with At Least K Repeating Characters	Return the length of the longest substring of `s` such that every character in it appears at least `k` times.	"At least k" is not monotone for a single window, so fix the number of distinct characters allowed. For each target `unique` from 1 to 26, run a max-window that holds exactly `unique` distinct chars, and record when all of them meet the count `k`.	O(26 * n)	O(1)	fix distinct count to restore monotonicity
487	Max Consecutive Ones II	Given a binary array `nums`, return the maximum number of consecutive 1s if you may flip at most one 0.	Identical to #1004 with `k = 1`: longest window containing at most one zero. Grow, and shrink only when a second zero enters.	O(n)	O(1)	at most one zero (k = 1)
689	Maximum Sum of 3 Non-Overlapping Subarrays	Find three non-overlapping subarrays of length `k` with maximum total sum and return their starting indices (lexicographically smallest on ties).	Precompute every window sum of size k, then for each middle window pick the best left window to its left and the best right window to its right.	O(n)	O(n)	middle window scan with k-gaps

#	Name	Description	Intuition	Time	Space	Key Implementation
713	Subarray Product Less Than K	Return the number of contiguous subarrays whose product of all elements is strictly less than `k`.	Max-variable window: grow the window while the product stays below k; every valid window ending at `j` contributes `j - i + 1` new subarrays.	O(n)	O(1)	count subarrays ending at j
727	Minimum Window Subsequence	Return the minimum-length substring (window) of `s1` such that `s2` is a subsequence of it. On ties, return the leftmost.	Walk forward matching `s2` as a subsequence; once fully matched at index `j`, walk backward to tighten the start, giving the smallest window ending at `j`. Restart the forward scan just past that start.	O(n * m)	O(1)	subsequence match, not contiguous
1044	Longest Duplicate Substring	Return any longest substring that appears at least twice in `s` (overlaps allowed); return `""` if none.	Binary search the answer length: if a duplicate of length `L` exists, a duplicate of any shorter length also exists. Check each candidate length with a rolling hash over a fixed-size window.	O(n log n) average	O(n)	binary search the window size
1838	Frequency of the Most Frequent Element	Given `nums` and `k` operations (each increments one element by 1), return the maximum possible frequency of any single value.	Sort the array; the cheapest target to raise a window of elements to is its maximum (the rightmost in a sorted window). A window `[i, j]` is achievable if raising all elements to `nums[j]` costs at most `k`.	O(n log n)	O(1)	sort so the window max is the cheapest target

Prefix Sum + HashMap `prefix_sum_hashmap.md` (6 problems)

#	Name	Description	Intuition	Time	Space	Key Implementation
560	Subarray Sum Equals K	Count the number of subarrays whose sum equals k.	A subarray summing to k ends here for every earlier prefix equal to `sum - k`; count those prefixes.	O(n)	O(n)	Standard
325	Maximum Size Subarray Sum Equals k	Find the length of the longest subarray summing to k.	Same complement as #560, but store the earliest index so `i - p` gives the longest subarray summing to k.	O(n)	O(n)	Standard
974	Subarray Sums Divisible by K	Count subarrays whose sum is divisible by k.	Two prefixes with the same remainder bracket a subarray divisible by k; count how many share each remainder.	O(n)	O(min(n, k))	Standard
523	Continuous Subarray Sum	Return true if any subarray of length ≥ 2 has sum divisible by k.	Same-remainder pair means a divisible subarray; store the earliest index so the gap `i - p >= 2` proves length ≥ 2.	O(n)	O(min(n, k))	Standard
437	Path Sum III	Count the number of paths in a binary tree that sum to targetSum. Paths can start and end at any node but must go downward.	Apply the #560 prefix-count idea along each root-to-node path; undo the prefix on the way back up so branches stay independent.	O(n)	O(h) (recursion + path-prefix map, h = tree height)	Standard
1	Two Sum	Given an array and a `target`, return the indices of the two numbers that add up to `target`.	Same "have I seen the complement before?" idea as prefix-sum, but the key is the raw value, so one pass finds the pair.	O(n)	O(n)	look up complement, not prefix sum

Binary Search `binary_search.md` (29 problems)

#	Name	Description	Intuition	Time	Space	Key Implementation
704	Binary Search	Given a sorted array and a target, return its index. Return -1 if not found.	The target is one specific value; halve a closed range until you sit on it (or the range empties).	O(log n)	O(1)	Standard
34	Find First and Last Position of Element in Sorted Array	Return `[first, last]` index of target. Return `{-1,-1}` if absent.	Find the boundary where "< target" flips to "≥ target" (first position), and the next boundary just past target.	O(log n)	O(1)	Standard
35	Search Insert Position	Return the index where target is or would be inserted to keep the array sorted.	The insert position is the first index whose value is ≥ target — exactly `lowerBound`.	O(log n)	O(1)	Standard
33	Search in Rotated Sorted Array	Array was sorted then rotated at an unknown pivot. Find target index, or -1.	Even rotated, one half is always sorted; check if target lies in that half and discard the other.	O(log n)	O(1)	Standard
162	Find Peak Element	Return any index where `arr[k] > arr[k-1]` and `arr[k] > arr[k+1]`. Edges are -∞.	Always walk uphill — an upward slope guarantees a peak to the right, so the slope direction is the "condition".	O(log n)	O(1)	Standard
875	Koko Eating Bananas	Minimum eating speed so Koko finishes all piles in h hours (one pile per hour, at most `speed` bananas eaten).	Feasibility is monotonic in speed (faster always works), so binary search the speed axis for the smallest that finishes in time.	O(n log(max(piles)))	O(1)	Standard
1011	Capacity to Ship Packages Within D Days	Minimum ship capacity to deliver all packages (in order) within d days.	Bigger capacity is always feasible, so binary search capacity for the smallest that ships within the day budget.	O(n log(sum(weights)))	O(1)	Standard
410	Split Array Largest Sum	Split array into m non-empty subarrays to minimize the largest subarray sum.	A larger allowed max-sum needs fewer parts, so binary search that cap for the smallest splittable into ≤ m parts.	O(n log(sum(nums)))	O(1)	Standard
1283	Find the Smallest Divisor Given a Threshold	Smallest positive divisor such that `sum of ceil(nums[i] / divisor) <= threshold`.	A bigger divisor shrinks the sum, so the condition flips false→true; binary search for the smallest divisor that fits the threshold.	O(n log(max(nums)))	O(1)	Standard
1231	Divide Chocolate ← MAXIMIZE (the one that differs)	Cut chocolate into k+1 pieces; keep the minimum-sweetness piece. Maximize that minimum.	A larger target-minimum yields fewer pieces, so the condition flips true→false; binary search for the largest minimum still giving ≥ k+1 pieces.	O(n log(sum(s)))	O(1)	Standard
4	Median of Two Sorted Arrays	Find the median of two sorted arrays of sizes m and n in O(log(m+n)) time.	Pick a split of the smaller array; the rest of the split is forced. Shrink the partition until the left half's max ≤ the right half's min, then the median falls out of the boundary values.	O(log(min(m, n)))	O(1)	search smaller array
69	Sqrt(x)	Compute the integer square root of a non-negative integer x without using sqrt().	The condition `k*k <= x` holds true...true then flips false as k grows; find the largest k still true. Ceiling mid avoids the infinite loop at `i+1 == j`.	O(log x)	O(1)	Standard
74	Search a 2D Matrix	Search for a target in an m×n matrix where each row is sorted and each row's first element > previous row's last.	The layout means the whole matrix reads as a single sorted sequence; map a flat index to `(row, col)` and run a standard closed binary search.	O(log(m*n))	O(1)	flatten index to 2D
81	Search in Rotated Sorted Array II	Search in a sorted, rotated array that may contain duplicates.	Duplicates can make both ends equal to the midpoint so neither half looks sorted; in that ambiguous case shrink both bounds by one and retry.	O(log n) average, O(n) worst	O(1)	ambiguous duplicates
153	Find Minimum in Rotated Sorted Array	Find the minimum element in a sorted, rotated array with no duplicates.	The minimum is the single point where the order breaks; if `arr[k] > arr[j]` the break is to the right, otherwise the min is at k or left.	O(log n)	O(1)	compare to right end
240	Search a 2D Matrix II	Search for a target in an m×n matrix where rows and columns are both sorted.	From the top-right, moving left strictly decreases and moving down strictly increases, so each comparison eliminates a full row or column.	O(m + n)	O(1)	start top-right corner
278	First Bad Version	Find the first bad version using a provided isBadVersion(n) API. Minimize API calls.	Versions are good...good then bad...bad once corrupted; this is the classic first-true boundary search.	O(log n)	O(1)	Standard
374	Guess Number Higher or Lower	Guess the number between 1 and n using a provided guess() API. Minimize calls.	`guess(k)` tells you which half holds the answer (`-1` lower, `1` higher); halve the closed range until it returns 0.	O(log n)	O(1)	Standard
475	Heaters	Given house and heater positions, find the minimum heater radius so every house is covered by some heater.	Each house needs the closest heater; the required radius is the largest of those closest distances. Find each house's nearest heater by binary search.	O((m + n) log m)	O(1)	also check left neighbor
540	Single Element in a Sorted Array	Find the single non-duplicate element in a sorted array where all others appear twice. O(log n).	Before the unique element each pair starts at an even index; after it, the parity shifts. Force the midpoint even and check whether its partner matches to pick a half.	O(log n)	O(1)	force k even
658	Find K Closest Elements	Find k integers in a sorted array closest to x, ordered by closeness then value.	The answer is a contiguous window of size k; search for its start by comparing the distances of the two window edges to x and sliding toward the closer side.	O(log(n - k) + k)	O(k)	search window start

#	Name	Description	Intuition	Time	Space	Key Implementation
719	Find K-th Smallest Pair Distance	Find the k-th smallest absolute distance among all pairs in the array.	The count of pairs with distance ≤ d is monotonic in d, so binary search the distance axis; count pairs ≤ d with a sliding window over the sorted array.	O(n log n + n log(maxDist))	O(1)	search distance value
852	Peak Index in a Mountain Array	Find the peak index in a mountain array (element greater than both neighbors).	An upward slope means the peak is strictly to the right; a downward slope means the peak is here or to the left.	O(log n)	O(1)	Standard
1060	Missing Element in Sorted Array	Find the kth missing number in a sorted array.	Missing count at each index increases monotonically, so binary search for the first index whose missing count is ≥ k, then offset from the previous element.	O(log n)	O(1)	kth missing beyond array end
1428	Leftmost Column with at Least a One	Find the leftmost column with at least one '1' in a binary matrix (rows sorted, accessed via BinaryMatrix API).	Each row is sorted 0...0 1...1; starting top-right, move left on a 1 (column might be even further left) and down on a 0, tracking the leftmost 1 seen.	O(rows + cols)	O(1)	start top-right corner
1482	Minimum Number of Days to Make m Bouquets	Find the minimum number of days to make m bouquets of k adjacent flowers.	Waiting more days only adds bloomed flowers, so feasibility is monotonic; binary search the day axis and greedily count adjacent runs of bloomed flowers.	O(n log(max(bloomDay)))	O(1)	impossible if not enough flowers
1539	Kth Missing Positive Number	Find the kth missing positive integer.	At each index the number of missing positives so far is `arr[k] - (k+1)`; binary search the first index whose missing count is ≥ k, then offset.	O(log n)	O(1)	i missing-free slots + k
1818	Minimum Absolute Sum Difference	Find the minimum absolute sum difference after one substitution.	Total cost is the sum of absolute differences; one swap can only help where the closest available nums1 value beats the current difference. Find that closest value by binary search and track the best gain.	O(n log n)	O(n)	also check left neighbor
1891	Cutting Ribbons	Find the maximum ribbon length such that m ribbons of that length can be cut.	Longer pieces produce fewer ribbons, so the count flips true→false as length grows; find the largest length still giving enough ribbons. Ceiling mid avoids the infinite loop.	O(n log(max(ribbons)))	O(1)	closed [i, j], result tracked

Sorting `sorting.md` (10 problems)

#	Name	Description	Intuition	Time	Space	Key Implementation
912	Sort an Array	Sort an integer array. No built-in sort allowed.	Divide and conquer — either split-then-merge (merge sort) or partition-then-recurse (quick sort); both reach O(n log n).	O(n log n)	O(n) merge / O(log n) quick	Standard
215	Kth Largest Element in an Array	Return the k-th largest element without sorting the full array.	Quick select partitions around a pivot and only recurses into the side containing the target rank, so it averages O(n) instead of fully sorting.	O(n) avg	O(log n)	Standard
973	K Closest Points to Origin	Return the k closest points to origin (0,0). Any order.	Same quick select, but partition on squared distance — once the first k slots are filled with the closest points, their internal order doesn't matter.	O(n) avg	O(log n)	Standard
148	Sort List	Sort a linked list in O(n log n) time, O(log n) space.	Merge sort fits linked lists — there's no random access for quick sort, but splitting at the middle and merging two sorted lists needs only pointer rewiring.	O(n log n)	O(log n)	Standard
179	Largest Number	Given a list of non-negative integers, arrange them to form the largest number.	Order two numbers by which concatenation is bigger (`b+a` vs `a+b`) — this pairwise rule sorts the whole list into the largest possible string.	O(n log n)	O(n)	Standard
315	Count of Smaller Numbers After Self	For each element, count how many elements to its right are smaller.	During a merge, whenever a left-half element is placed, every right-half element already emitted is both smaller and to its right — so the merge counts the smaller-after-self pairs for free.	O(n log n)	O(n)	Standard
56	Merge Intervals	Merge all overlapping intervals. Return array of non-overlapping intervals.	After sorting by start, overlaps can only be with the most recent interval, so one scan either extends its end or appends a new one.	O(n log n)	O(n)	Standard
88	Merge Sorted Array	Given two sorted arrays `nums1` (with capacity for m+n) and `nums2` , merge `nums2` into `nums1` in-place.		O(m+n)	O(1)	Fill from the END backwards using three pointers `i` (end of nums1 valid data), `j` (end of nums2), `k` (end of merged result). This avoids the need to shift elements.
327	Count of Range Sum	Given an integer array, count the number of range sums `sum(i,j)` that lie within `[lower, upper]` (inclusive).		O(n log n)	O(n)	Build prefix sum array. Then use modified merge sort — during the merge phase, for each element in the left half, use two sliding pointers `j` and `k` into the right half to count valid prefix sums (those where `arr[k] - arr[i]` falls in range). This gives O(n log n) vs O(n^2) brute force.
164	Maximum Gap	Given an unsorted array, return the maximum difference between successive elements in its sorted form. Must run in linear time.		O(n)	O(n)	bucket sort by pigeonhole principle. With n elements, the max gap is at least `ceil((max-min)/(n-1))` . Place elements into buckets of that size; the max gap spans between one bucket's max and the next non-empty bucket's min.

Linked List `linked_list.md` (32 problems)

#	Name	Description	Intuition	Time	Space	Key Implementation
203	Remove Linked List Elements	Remove all nodes whose value equals `val`.		O(n)	O(1)	Standard
83	Remove Duplicates from Sorted List	Keep exactly one copy of each value. List is sorted.				Standard
82	Remove Duplicates from Sorted List II	Remove ALL nodes that have duplicate values — only distinct-valued nodes remain.		O(n)	O(1)	Standard
19	Remove Nth Node From End of List	Remove the nth node from the end. Return the head.				Standard
206	Reverse Linked List	Reverse the entire linked list. Return the new head.	flip each node's arrow to point at the node you just came from; `previous` is the reversed list built so far.			Standard
92	Reverse Linked List II	Reverse only the nodes from position `left` to `right` (1-indexed).				Standard
25	Reverse Nodes in k-Group	Reverse every consecutive group of k nodes. Leave remaining nodes (< k) as-is.		O(n)	O(1)	Standard
876	Middle of Linked List	Return the middle node. For even length, return the second middle.		O(n)	O(1)	Standard
141	Linked List Cycle	Return true if the list contains a cycle.		O(n)	O(1)	Standard
142	Linked List Cycle II	Return the node where the cycle begins. Return null if no cycle.	the distance from head to the cycle entry equals the distance from the meeting point to the entry, so two 1-step walkers from those spots collide exactly at the entry.	O(n)	O(1)	Standard
21	Merge Two Sorted Lists	Merge two sorted linked lists into one sorted list.	like a zipper — always splice on whichever current head is smaller, then advance that list.	O(n)	O(1)	Standard
23	Merge K Sorted Lists	Merge k sorted linked lists into one sorted list.		O(n log k)	O(k)	Standard
143	Reorder List	Reorder list as L0 → Ln → L1 → Ln-1 → L2 → ...				Standard
328	Odd Even Linked List	Group all odd-indexed nodes first, then even-indexed nodes. Preserve relative order within each group. (Indexing is 1-based.)				Standard
2	Add Two Numbers	Two non-empty linked lists represent non-negative integers stored in reverse order. Add and return the sum as a linked list in reverse order.		O(max(m, n))	O(max(m, n))	loop until carry clears
88	Merge Sorted Array	Merge `nums2` into `nums1` in-place. `nums1` has enough space at the end. Both arrays are sorted.		O(m + n)	O(1)	three pointers from END
234	Palindrome Linked List	Check if a singly linked list is a palindrome in O(n) time and O(1) space.		O(n)	O(1)	in-place reverse
160	Intersection of Two Linked Lists	Find the node at which two singly linked lists intersect. Return null if they do not intersect.		O(m + n)	O(1)	redirect to other list on null
24	Swap Nodes in Pairs	Swap every two adjacent nodes in a linked list and return the modified head.	A sentinel removes head special-casing. For each pair, `previous` is the node before the pair; re-wire the three pointers so the second node leads, then advance `previous` two nodes forward.	O(n)	O(1)	Standard
61	Rotate List	Rotate a linked list to the right by `k` places.	Rotating right by `k` is equivalent to making the list circular, then breaking it `k % length` nodes before the old tail. Compute the length, close the ring, then walk to the new tail.	O(n)	O(1)	only the remainder matters
86	Partition List	Partition a linked list so all nodes with value `< x` come before nodes with value `>= x`. Preserve the relative order within each group.	Build two separate chains with two sentinels — one for the "less" group and one for the "greater-or-equal" group — then stitch them together. Preserving original order falls out naturally because nodes are appended in traversal order.	O(n)	O(1)	Standard

#	Name	Description	Intuition	Time	Space	Key Implementation
109	Convert Sorted List to Binary Search Tree	Convert a sorted linked list to a height-balanced BST.	A sorted list is an in-order traversal of a BST. Repeatedly pick the middle node as the subtree root so each side gets a balanced number of nodes; recurse on the left half (before middle) and right half (after middle).	O(n log n)	O(log n)	stop at exclusive tail
138	Copy List with Random Pointer	Deep-copy a linked list where each node has a `random` pointer to any node or null.	Interleave each clone directly after its original so a clone's `random` is reachable as `original.random.next` — no hash map needed. Three passes: weave clones in, wire random pointers, then unweave.	O(n)	O(1)	Standard
148	Sort List	Sort a linked list. Aim for O(n log n) time and O(1) extra space (top-down recursion uses O(log n) stack).	Merge sort fits linked lists perfectly — splitting is a pointer cut and merging reuses the #21 zipper. Find the middle, sort each half, merge.	O(n log n)	O(log n)	Standard
237	Delete Node in a Linked List	Delete a node from a singly linked list given only a reference to that node (not the head). The node is guaranteed not to be the tail.	Without access to the previous node you cannot unlink the given node directly, but you can copy the next node's value into it and then delete the next node instead.	O(1)	O(1)	copy successor's value
369	Plus One Linked List	A non-negative integer is stored as a linked list of digits in big-endian order (most significant first). Add one to it and return the resulting list.	Reverse the list so addition flows least-significant-first like normal carry arithmetic, add one with carry propagation, then reverse back. A possible new leading digit is handled by extending the tail after reversal.	O(n)	O(1)	seed carry with the +1
430	Flatten a Multilevel Doubly Linked List	Flatten a multilevel doubly linked list where nodes may have a `child` list. Produce a single-level list using the `next` pointers; all `child` pointers must be null.	A child list should be spliced in immediately after its parent, before the parent's original `next`. A stack lets you remember each deferred `next` while you dive into a child branch — effectively an iterative DFS.	O(n)	O(n)	clear child after splicing
445	Add Two Numbers II	Two numbers are stored as linked lists in big-endian order (most significant digit first). Add them and return the sum as a linked list, also in big-endian order.	Addition needs least-significant digits first, but reversing the inputs is discouraged here. Push all digits onto two stacks; popping yields least-significant-first. Build the result by prepending nodes so the final list is big-endian.	O(m + n)	O(m + n)	prepend to keep big-endian order
708	Insert into a Sorted Circular Linked List	Insert a value into a sorted circular linked list and return a reference to any node in the list. The given pointer may reference any node, or be null for an empty list.	Walk one full loop looking for the correct gap between a node and its successor. Three cases cover it: a normal in-between slot, the wrap-around boundary (max→min) where the value is an extreme, and a uniform-value list where any spot works.	O(n)	O(1)	wrap-around boundary
725	Split Linked List in Parts	Split a linked list into `k` consecutive parts as equal in size as possible. Earlier parts have sizes greater than or equal to later parts; some trailing parts may be empty.	With length `n`, each part gets `n / k` nodes, and the first `n % k` parts get one extra. Walk through cutting off each part at its computed size.	O(n)	O(k)	first `remainder` parts get +1
1171	Remove Zero Sum Consecutive Nodes from Linked List	Repeatedly remove consecutive sequences of nodes that sum to zero until no such sequence remains. Return the resulting list.	Use prefix sums. If two nodes share the same running prefix sum, every node strictly between them sums to zero and can be dropped. A hash map from prefix sum to node lets you splice out those runs in one pass.	O(n)	O(n)	keep the latest node per sum
1265	Print Immutable Linked List in Reverse	Print all values of an immutable linked list in reverse, using only the exposed `getNext()` and `printValue()` API, without modifying the list.	Recursion naturally reverses order — recurse to the end first, then print on the way back up the call stack.	O(n)	O(n)	recurse first, print on unwind

String `string.md` (87 problems)

#	Name	Description	Intuition	Time	Space	Key Implementation
242	Valid Anagram	Return true if `t` is an anagram of `s` (same characters, same counts).	Anagrams have identical letter counts, so add up `s` and subtract `t` in one 26-slot array — all zeros means they match.	O(n)	O(1)	Standard
383	Ransom Note	Can `ransomNote` be constructed from the letters in `magazine` (each letter used at most once)?	Stock the letter counts from the magazine, then draw down for each note letter — if any count goes negative the magazine is short.	O(n)	O(1)	Standard
387	First Unique Character in a String	Return the index of the first non-repeating character. Return -1 if none exists.	One pass to count every letter, a second pass to return the first whose count is exactly 1.	O(n)	O(1)	Standard
49	Group Anagrams	Group strings that are anagrams of each other.	Anagrams share identical letter frequencies, so the frequency vector encoded as a string is a unique group key — O(k) to build vs O(k log k) to sort the characters.	O(nk)	O(n)	Standard
451	Sort Characters By Frequency	Sort characters in descending order of frequency. Ties can go in any order.	Frequencies are bounded by string length, so bucket chars by their count and read buckets high-to-low — no O(n log n) comparison sort needed.			Standard
5	Longest Palindromic Substring	Return the longest palindromic substring.	Every palindrome has a center; try all 2n-1 centers (each char and each gap) and grow outward while characters mirror.	$O(n^2)$	O(1)	Standard
647	Palindromic Substrings	Count all palindromic substrings.	Same center-expansion, but each successful step outward is itself one more palindrome, so accumulate a count rather than a max.	$O(n^2)$	O(1)	Standard
8	String to Integer (atoi)	Parse a string to an integer, handling leading spaces, optional sign, overflow, and non-digit stop.	Process the string in strict phases — spaces, then sign, then digits accumulated as `num*10 + digit` — clamping to int bounds the moment overflow would occur.	O(n)	O(1)	Standard
14	Longest Common Prefix	Find the longest common prefix string among an array of strings.	Start with the first string as the candidate prefix and trim its tail until every other string starts with it.	O(n·k)	O(1)	Standard
443	String Compression	Compress the char array in-place. Each group `aaa` becomes `a3`. Single chars stay as-is. Return new length.	Two pointers — read scans each run while write lays down the compressed `char + count` behind it; write never overtakes read, so it's safe in place.	O(n)	O(1)	Standard
1047	Remove All Adjacent Duplicates in String	Repeatedly remove adjacent equal characters until no more exist.	A stack mirrors the partially-cleaned string — if the next char equals the top it's an adjacent pair, so pop instead of pushing.	O(n)	O(n)	Standard
6	ZigZag Conversion	Convert a string into a zigzag pattern across `numRows` rows and read it row by row.	Walk the string once, appending each char to its current row's builder while bouncing the row index down then up.	O(n)	O(n)	Standard
12	Integer to Roman	Convert an integer to its Roman numeral representation (values 1–3999).	Greedily subtract the largest possible value (including the 4/9 subtractive pairs) and append its symbol.	O(1)	O(1)	Standard
13	Roman to Integer	Convert a Roman numeral string to an integer.	Add each symbol's value, but subtract instead when a smaller value precedes a larger one (subtractive notation).	O(n)	O(1)	Standard
28	Implement strStr()	Find the first occurrence of `needle` in `haystack`. Return -1 if not found.	Slide a window of needle's length across haystack and compare; the first full match wins.	O(n·m)	O(1)	Standard
38	Count and Say	Generate the nth term of the "count and say" sequence: read digits of the previous term aloud.	Starting from "1", repeatedly scan runs of equal digits and emit count followed by the digit.	O(n·m)	O(m)	Standard
43	Multiply Strings	Multiply two non-negative integers given as strings. Return the product as a string.	Schoolbook multiplication: digit i of num1 times digit j of num2 lands in result positions i+j and i+j+1.	O(n·m)	O(n+m)	Standard
58	Length of Last Word	Return the length of the last word in a string (word = max sequence of non-space chars).	Scan from the right: skip trailing spaces, then count letters until the next space.	O(n)	O(1)	Standard
65	Valid Number	Validate whether a string is a valid decimal number (integers, fractions, exponents).	A single left-to-right scan tracking whether we have seen a digit, a dot, and an exponent enforces all the ordering rules.	O(n)	O(1)	Standard
68	Text Justification	Format words into lines of `maxWidth`, fully justified (equal spaces, last line left-aligned).	Greedily pack as many words as fit per line, then distribute leftover spaces evenly between gaps (extras to the left); the last line is left-justified.	O(n·maxWidth)	O(n)	Standard
125	Valid Palindrome	Check if a string is a palindrome, ignoring non-alphanumeric characters and case.	Two pointers move inward, skipping non-alphanumeric chars and comparing lowercased letters.	O(n)	O(1)	Standard
151	Reverse Words in a String	Reverse the order of words in a string (split on spaces, handle multiple spaces).	Scan from the right, extract each word, and append them in reverse order with single spaces.	O(n)	O(n)	Standard
161	One Edit Distance	Determine if two strings are one edit distance apart (insert, delete, or replace one char).	Scan to the first mismatch; equal lengths force a replace, otherwise skip one char in the longer string and require the rest to match.	O(n)	O(n)	Standard
163	Missing Ranges	Given a sorted array, return missing ranges between `lower` and `upper` bounds.	Walk a running "next expected" value; whenever a number exceeds it, the gap before it is a missing range.	O(n)	O(1)	Standard
165	Compare Version Numbers	Compare two version number strings (dot-separated integers). Return -1, 0, or 1.	Parse both revisions position by position, treating missing components as 0, and compare numerically.	O(n)	O(n)	Standard

#	Name	Description	Intuition	Time	Space	Key Implementation
166	Fraction to Recurring Decimal	Convert a fraction numerator/denominator to a decimal string, noting repeating parts in parentheses.	Do long division; remember the position of each remainder so a repeat reveals the cycle to wrap in parentheses.	O(den)	O(den)	Standard
179	Largest Number	Arrange numbers so their concatenation forms the largest possible number.	Sort by a custom comparator that prefers the order producing a larger concatenation (a+b vs b+a).	O(n log n · k)	O(n)	Standard
186	Reverse Words in a String II	Reverse words in a character array in-place (words separated by spaces).	Reverse the whole array, then reverse each word back so the order flips but each word reads forward.	O(n)	O(1)	Standard
205	Isomorphic Strings	Check if two strings follow the same character-substitution pattern (isomorphic).	Maintain a bijective mapping both ways; any conflicting mapping breaks isomorphism.	O(n)	O(1)	Standard
214	Shortest Palindrome	Find the shortest palindrome by adding characters in front of a string.	Find the longest palindromic prefix via KMP failure on `s + '#' + reverse(s)`; reverse the leftover suffix and prepend it.	O(n)	O(n)	Standard
228	Summary Ranges	Given a sorted unique integer array, summarize consecutive runs as ranges.	Track the start of each consecutive run; when the run breaks, emit either a single number or a "start->end" range.	O(n)	O(1)	Standard
246	Strobogrammatic Number	Check if a number reads the same when rotated 180 degrees.	Two pointers move inward; each pair must be a valid strobogrammatic mapping (0-0, 1-1, 6-9, 8-8, 9-6).	O(n)	O(1)	Standard
249	Group Shifted Strings	Group strings that follow the same shift sequence (each char shifted by the same amount).	Encode each string by the gaps between consecutive chars (mod 26) so all members of a shift group share the same key.	O(n·k)	O(n·k)	Standard
266	Palindrome Permutation	Check if a string can be rearranged into a palindrome (at most one odd-count char).	A palindrome allows at most one character with an odd count; track parity with a set.	O(n)	O(1)	Standard
271	Encode and Decode Strings	Encode a list of strings to a single string and decode it back.	Length-prefix each string ("len#str") so the decoder always knows exactly how many chars to read, regardless of content.	O(n)	O(n)	Standard
273	Integer to English Words	Convert a non-negative integer to its English words representation.	Process the number in groups of three digits, naming each group and appending the right scale word (Thousand, Million, Billion).	O(1)	O(1)	Standard
290	Word Pattern	Check if a string `s` follows a given pattern (bijective character-to-word mapping).	Maintain a two-way mapping between pattern chars and words; any conflict breaks the bijection.	O(n)	O(n)	Standard
299	Bulls and Cows	Count bulls (right digit, right place) and cows (right digit, wrong place) in a number guessing game.	Count exact-position matches as bulls; for the rest, use a digit-count array where positive entries (secret surplus) and negative entries (guess surplus) reveal cows.	O(n)	O(1)	Standard
344	Reverse String	Reverse a character array in-place.	Two pointers swap from both ends moving inward.	O(n)	O(1)	Standard
345	Reverse Vowels of a String	Reverse only the vowels of a string.	Two pointers advance until each lands on a vowel, then swap those vowels.	O(n)	O(n)	Standard
388	Longest Absolute File Path	Find the length of the longest absolute file path in a file system string.	Tab depth gives the nesting level; keep a stack of path lengths per level and, on hitting a file (has a dot), compute the full path length.	O(n)	O(n)	Standard
392	Is Subsequence	Check if string `s` is a subsequence of string `t`.	Walk `t` with one pointer; advance the `s` pointer each time chars match. If `s` is exhausted, it is a subsequence.	O(n)	O(1)	Standard
408	Valid Word Abbreviation	Check if an abbreviation matches a word (digits represent skipped characters).	Walk both with pointers; on a digit, parse the skip count (no leading zero) and jump the word pointer; otherwise chars must match.	O(n)	O(1)	Standard
409	Longest Palindrome	Find the length of the longest palindrome that can be built from the given letters.	Use every pair of each letter; if any letter has a leftover single, one of them can sit in the center.	O(n)	O(1)	Standard
415	Add Strings	Add two non-negative integers given as strings. Return the sum as a string.	Add digit by digit from the right with a carry, exactly like grade-school addition.	O(max(n,m))	O(max(n,m))	Standard
422	Valid Word Square	Given a sequence of words, check whether they form a valid word square (kth row equals kth column).	For each cell (i,j), the char must equal the char at (j,i); guard against ragged rows by bounds-checking.	O(n·m)	O(1)	Standard
423	Reconstruct Original Digits from English	Given a scrambled string of digit-word letters, decode the original digits in order.	Some letters are unique to one digit (z→zero, w→two, u→four, x→six, g→eight); count those first, then peel off the remaining digits using letters that become unique after subtraction.	O(n)	O(1)	Standard
434	Number of Segments in a String	Count the number of segments (maximal runs of non-space chars) in a string.	A new segment starts at each non-space char whose predecessor is a space (or string start).	O(n)	O(1)	Standard
459	Repeated Substring Pattern	Check if a string can be built by repeating one of its substrings multiple times.	If `s` is a repeated pattern, it appears inside `(s+s)` with the first and last char removed.	O(n)	O(n)	Standard
468	Validate IP Address	Validate whether a string is a valid IPv4 or IPv6 address.	Dispatch on the separator: dots mean IPv4 (four 0–255 octets, no leading zeros), colons mean IPv6 (eight 1–4 hex groups).	O(n)	O(n)	Standard
500	Keyboard Row	Find all words that can be typed on a single row of a QWERTY keyboard.	Map each letter to its row index, then keep words whose letters all share one row.	O(n·k)	O(n)	Standard
520	Detect Capital	Check if a word is capitalized correctly (all caps, all lower, or first letter only).	Count uppercase letters: valid only if all are uppercase, none are, or exactly the first one is.	O(n)	O(1)	Standard

#	Name	Description	Intuition	Time	Space	Key Implementation
524	Longest Word in Dictionary through Deleting	Find the longest word in a dictionary that is a subsequence of string `s` (smallest lexicographically on ties).	Check each candidate with the subsequence two-pointer, keeping the best by length then lexicographic order.	O(n·k)	O(1)	Standard
541	Reverse String II	Reverse the first `k` characters of every `2k`-character block in a string.	Step through the array in `2k` jumps, reversing the first `k` chars of each block (or all that remain).	O(n)	O(n)	Standard
551	Student Attendance Record I	Check if a record is award-eligible (fewer than 2 absences total, no 3 consecutive lates).	Count total absences and track the current run of lates; fail on the second absence or third consecutive late.	O(n)	O(1)	Standard
556	Next Greater Element III	Find the next greater number by rearranging the digits of `n`. Return -1 if none or on overflow.	Standard next-permutation on digits: find the rightmost ascending pair, swap with the next larger digit to its right, reverse the suffix.	O(d)	O(d)	Standard
557	Reverse Words in a String III	Reverse individual words in a string while preserving word order.	Split on spaces, reverse each word, and rejoin with single spaces.	O(n)	O(n)	Standard
592	Fraction Addition and Subtraction	Add or subtract a sequence of fractions, returning the result in lowest terms.	Parse each signed fraction, accumulate over a common denominator, then reduce by the GCD.	O(n)	O(1)	Standard
616	Add Bold Tag in String	Wrap every substring of `s` that matches a word in the dictionary with `<b>...</b>`, merging overlaps.	Mark every covered index in a boolean array, then emit bold tags around maximal marked runs.	O(n·m)	O(n)	Standard
657	Robot Return to Origin	Check if a robot following an instruction string (UDLR) returns to the origin.	Track net horizontal and vertical displacement; the robot returns only if both are zero.	O(n)	O(1)	Standard
670	Maximum Swap	Swap two digits at most once to get the maximum possible value.	Record the last index of each digit; scanning left to right, swap the first digit with the largest later digit that exceeds it.	O(d)	O(1)	Standard
680	Valid Palindrome II	Check if a string is a palindrome after deleting at most one character.	Two pointers inward; at the first mismatch, try skipping either the left or right char and check the remainder.	O(n)	O(1)	Standard
681	Next Closest Time	Find the next closest valid time using only the digits present in a given time string.	From the current minute, advance one minute at a time and return the first time whose digits are all in the allowed set.	O(1)	O(1)	Standard
686	Repeated String Match	Find the minimum number of times string `a` must repeat so `b` is a substring. Return -1 if impossible.	Repeat `a` until its length covers `b`, then check one extra copy to handle offset overlap.	O(n·m)	O(n)	Standard
709	To Lower Case	Convert a string to lowercase.	Uppercase ASCII letters differ from lowercase by 32, so shift any A–Z char.	O(n)	O(n)	Standard
726	Number of Atoms	Parse a chemical formula string and return atom counts in sorted order.	Recursive-descent style parse with a stack of count maps for parentheses; multiply a group's counts by the number following its closing paren.	O(n)	O(n)	Standard
748	Shortest Completing Word	Find the shortest word in a list that contains all letters of a license plate (case-insensitive, with multiplicity).	Build the plate's letter-count vector, then keep the shortest word whose own counts cover it.	O(n·k)	O(1)	Standard
788	Rotated Digits	Count numbers in `[1, n]` that become a different valid number when each digit is rotated 180 degrees.	A number is "good" if all digits are valid rotations and at least one digit (2,5,6,9) actually changes.	O(n·d)	O(1)	Standard
791	Custom Sort String	Sort string `s` so its characters appear in the order given by string `order`.	Count chars in `s`, emit them in `order`'s sequence, then append any chars not mentioned in `order`.	O(n)	O(n)	Standard
794	Valid Tic-Tac-Toe State	Decide whether the given tic-tac-toe board is reachable from valid play.	Count X and O; X count must equal O or be one more, and if a player has won the move counts must be consistent (both can't win).	O(1)	O(1)	Standard
796	Rotate String	Check if string `t` is a rotation of string `s`.	Every rotation of `s` is a substring of `s+s`, so check containment after a length match.	$O(n^2)$	O(n)	Standard
809	Expressive Words	Determine how many query words match a stretched version of `s` (groups stretched to length ≥ 3).	Compare run lengths group by group: the stretched group must be ≥ 3, or equal to the original run length.	O(n·k)	O(1)	Standard
824	Goat Latin	Convert words to Goat Latin: append "ma" (plus per-word index of 'a's); move leading consonants to the end for non-vowel words.	For each word, decide vowel vs consonant start, transform, then append "ma" and i+1 trailing 'a's.	O(n)	O(n)	Standard
833	Find And Replace in String	Apply non-overlapping indexed replacements: at each index, if `sources[k]` matches, replace it with `targets[k]`.	Map each start index to its replacement; rebuild the string, jumping over matched sources and copying unmatched chars verbatim.	O(n)	O(n)	Standard
859	Buddy Strings	Check if two strings are "buddy strings": swapping exactly one pair of chars makes them equal.	Equal length is required; if the strings are identical, you need a duplicate char to swap; otherwise exactly two positions must differ and be mirror images.	O(n)	O(1)	Standard
953	Verifying an Alien Dictionary	Check if words are sorted in a given alien language's alphabetical order.	Map each letter to its rank, then verify each adjacent pair is in non-decreasing order under that ranking.	O(n·k)	O(1)	Standard
1055	Shortest Way to Form String	Find the minimum number of subsequences of `source` whose concatenation equals `target`. Return -1 if impossible.	Greedily consume as much of `target` as possible from each pass over `source`; if a pass makes no progress, a needed char is missing.	O(n·m)	O(1)	Standard

#	Name	Description	Intuition	Time	Space	Key Implementation
1108	Defanging an IP Address	Defang an IP address by replacing each '.' with '[.]'.	Append each char, expanding dots into the bracketed form.	O(n)	O(n)	Standard
1221	Split a String in Balanced Strings	Find the maximum number of balanced strings ('R' count == 'L' count) to split into.	Track a running balance; every time it returns to zero a balanced piece has closed.	O(n)	O(1)	Standard
1328	Break a Palindrome	Break a palindrome by changing one character to get the lexicographically smallest non-palindrome.	Change the first non-'a' in the first half to 'a'; if all are 'a', change the last char to 'b'. Single-char strings can't be broken.	O(n)	O(n)	Standard
1392	Longest Happy Prefix	Find the longest happy prefix (a non-empty prefix that is also a suffix).	The KMP failure value at the last index is exactly the length of the longest proper prefix that is also a suffix.	O(n)	O(n)	Standard
1446	Consecutive Characters	Find the maximum number of consecutive same characters in a string (the "power" of the string).	Track the current run length, resetting on a change, and keep the maximum seen.	O(n)	O(1)	Standard
1554	Strings Differ by One Character	Check if any two strings in a list differ by exactly one character (same length, same position).	For each position, hash each word with that position wildcarded; a collision among same-length words means they differ in only that spot.	$O(n \cdot m^2)$	$O(n \cdot m)$	Standard
1556	Thousand Separator	Format an integer with a dot as the thousands separator.	Build the result from the right, inserting a separator after every three digits.	O(d)	O(d)	Standard
1736	Latest Time by Replacing Hidden Digits	Find the latest valid time by replacing each '?' with an appropriate digit.	Greedily choose the largest valid digit at each '?' position, respecting hour (≤ 23) and minute (≤ 59) constraints.	O(1)	O(1)	Standard
1816	Truncate Sentence	Truncate a sentence to its first `k` words.	Copy chars until the (k)th space is reached.	O(n)	O(n)	Standard
1844	Replace All Digits with Characters	Replace each digit at an odd index with the letter shifted from the preceding char by that digit.	Even indices are letters; for each odd index, shift the previous letter forward by the digit's value.	O(n)	O(n)	Standard

Stack / Queue / Heap `stack_queue.md` (58 problems)

#	Name	Description	Intuition	Time	Space	Key Implementation
20	Valid Parentheses	Given a string of brackets `()[]{}`, return true if all brackets are properly closed and ordered.	the most recent unmatched open bracket is the only one a closing bracket can legally match — that is exactly LIFO.	O(n)	O(n)	Standard
155	Min Stack	Design a stack that supports push, pop, top, and `getMin()` in O(1).		O(1) per operation	O(n)	Standard
394	Decode String	Decode a string like `"3[a2[bc]]"` → `"abcbcabcbc"`.		O(n) (n = length of decoded output)	O(n)	Standard
739	Daily Temperatures	For each day, how many days until a warmer temperature? Return 0 if none.	a colder day just waits on the stack until the first warmer day arrives and resolves it.	O(n)	O(n)	Standard
496	Next Greater Element I	For each element in `nums1` (a subset of `nums2`), find the first greater element to its right in `nums2`. Return -1 if none.		O(m + n)	O(n)	Standard
84	Largest Rectangle in Histogram	Find the largest rectangle that can be formed within the histogram bars.	a bar can only stretch sideways until it hits a shorter bar; the stack remembers each bar's left limit so the right limit (current shorter bar) closes the rectangle.	O(n)	O(n)	Standard
42	Trapping Rain Water	How much water can be trapped between the bars after rain?	water sits in a dip between a left wall and a right wall, and its depth is set by whichever wall is shorter.	O(n)	O(n)	Standard
239	Sliding Window Maximum	Return the maximum in each sliding window of size k.	any element smaller than a newer element can never be the window max again, so discard it; the front always holds the current best.	O(n)	O(k)	Standard
1046	Last Stone Weight	Smash the two heaviest stones repeatedly. Return the last remaining weight (or 0).		O(n log n)	O(n)	Standard
347	Top K Frequent Elements	Return the k most frequent elements.		O(n log k)	O(n)	Standard
295	Find Median from Data Stream	Add integers to a stream and find the median at any time in O(log n) add and O(1) find.	keep the smaller half and larger half balanced; the median always lives right at the boundary, on the tops of the two heaps.	O(log n) addNum, O(1) findMedian	O(n)	Standard
502	IPO	Before each project you must have enough capital. Start with capital `w`. Pick at most `k` projects to maximize final capital.		O(n log n)	O(n)	Standard
767	Reorganize String	Rearrange characters so no two adjacent characters are the same. Return `""` if impossible.		O(n log k) (k = distinct chars ≤ 26)	O(k)	Standard
503	Next Greater Element II	Given a circular integer array, return the next greater number for every element. The next greater number of an element is the first greater number found by traversing the array circularly.		O(n)	O(n)	Monotone decreasing stack. Iterate through the array TWICE (indices 0 to 2n-1, using `i % n`). Only push index `i` onto the stack during the first pass (`i < n`) to avoid duplicates.
85	Maximal Rectangle	Given a binary matrix filled with 0s and 1s, find the largest rectangle containing only 1s and return its area.		O(m·n)	O(n)	For each row, compute cumulative histogram heights (1s stack vertically). Then apply the Largest Rectangle in Histogram algorithm (#84) on each row's histogram.
224	Basic Calculator	Evaluate a string expression containing `+`, `-`, `(`, `)`, digits, and spaces.		O(n)	O(n)	When encountering `(`, push current (result, sign) onto stack and reset. When encountering `)`, combine current result with the saved result and sign from the stack.
227	Basic Calculator II	Evaluate a string expression with `+`, `-`, `*`, `/` and integers (no parentheses).		O(n)	O(n)	Process operator BEFORE the current number. Push positive/negative numbers for `+` / `-`. For `*` / `/`, immediately multiply/divide the top of the stack. Sum the stack at the end.

#	Name	Description	Intuition	Time	Space	Key Implementation
378	Kth Smallest Element in a Sorted Matrix	Given an n×n matrix where each row and column is sorted in ascending order, return the kth smallest element.		O(n log(max-min))	O(1)	Binary search on the VALUE range [matrix[0][0], matrix[n-1][n-1]]. For a given `mid`, count elements ≤ mid by scanning from the bottom-left corner: if `matrix[row][col] <= mid`, all `row+1` elements in this column (0..row) are ≤ mid.
373	Find K Pairs with Smallest Sums	Given two sorted arrays `nums1` and `nums2`, return the `k` pairs `(u, v)` (one from each) with the smallest sums.		O(k log k)	O(k)	Seed min-heap with `(nums1[i], nums2[0])` for i = 0..min(k, n1.length)-1. On each pop, if `j+1 < nums2.length`, push `(nums1[i], nums2[j+1])`. This expands row-by-row in the implicit pair matrix.
621	Task Scheduler	Given a list of tasks (characters) and a cooldown `n`, return the minimum number of intervals to finish all tasks. Between two same tasks there must be at least `n` intervals.		O(k)	O(1)	Greedy formula. Find the most frequent task (maxFreq). It creates `maxFreq − 1` "frames" of `n + 1` slots, plus `maxCount` (count of tasks with maxFreq) slots. The answer is `max(tasks.length, (maxFreq − 1) ∗ (n + 1) + maxCount)`.
358	Rearrange String k Distance Apart	Rearrange a string so that the same characters are at least `k` distance apart. Return "" if impossible.		O(n log k)	O(k)	greedy with a max-heap by frequency plus a cooldown queue of size k that holds recently used characters until they're eligible again.
480	Sliding Window Median	Return the median of every window of size k as it slides across the array.		O(n·k) with heap remove (O(n log k) with indexed sets)	O(k)	two heaps (maxHeap = lower half, minHeap = upper half) with lazy deletion — defer removing elements that left the window, tracking balance with counts.
692	Top K Frequent Words	Return the k most frequent words. Sort by frequency descending; ties broken by lexicographic order.		O(n log k)	O(n)	min-heap of size k ordered so the "smallest" (lowest freq, or same freq with lexicographically larger word) sits on top for eviction.
772	Basic Calculator III	Evaluate an arithmetic expression with `+`, `−`, `∗`, `/`, and parentheses.		O(n)	O(n)	combines #224 (parentheses via recursion) and #227 (operator precedence: apply `∗` / `/` immediately, defer `+` / `−`).
32	Longest Valid Parentheses	Find the length of the longest valid (well-formed) parentheses substring.	push indices onto a stack; the bottom of the stack is always the index just before the current valid run, so the gap to it gives the run length.	O(n)	O(n)	Standard
71	Simplify Path	Simplify an absolute Unix file path (handle `.`, `..`, multiple slashes).	split on `/` and treat directories as a stack — `..` pops the last directory, `.` and empty tokens are ignored.	O(n)	O(n)	Standard
150	Evaluate Reverse Polish Notation	Evaluate a Reverse Polish Notation expression with `+`, `−`, `∗`, `/`.	push operands; on an operator pop the two most recent operands, apply, and push the result back — exactly LIFO.	O(n)	O(n)	Standard
215	Kth Largest Element in an Array	Find the kth largest element in an unsorted array (not necessarily distinct).	a min-heap of size k keeps the k largest seen so far; its top is the kth largest.	O(n log k)	O(k)	Standard
218	The Skyline Problem	Given building rectangles, return the skyline as a list of key points.	sweep left to right over building edges; a max-heap of active heights tracks the tallest standing building, and a key point is emitted whenever that maximum changes.	O(n^2) (heap remove)	O(n)	Standard
225	Implement Stack using Queues	Implement a LIFO stack using only queue operations.	after each push, rotate the queue so the newest element sits at the front — then `poll` / `peek` behave like stack `pop` / `top`.	O(n) push, O(1) others	O(n)	Standard
232	Implement Queue using Stacks	Implement a queue using two stacks.	one stack receives pushes, the other serves pops; moving elements over reverses their order, restoring FIFO. Amortized O(1) per element.	O(1) amortized	O(n)	two stacks emulate FIFO
316	Remove Duplicate Letters	Remove duplicate letters so each appears once, result is lexicographically smallest.	monotone increasing stack of letters — pop a larger letter when a smaller one arrives, but only if the popped letter appears again later (so we can re-add it).	O(n)	O(1) (26 letters)	Standard
402	Remove K Digits	Remove k digits from a number string to get the smallest possible number.	monotone increasing stack of digits — whenever a smaller digit arrives, pop larger preceding digits (each pop is one removal) so high places hold the smallest digits.	O(n)	O(n)	Standard

#	Name	Description	Intuition	Time	Space	Key Implementation
456	132 Pattern	Check if a 1-3-2 pattern exists in an integer array.		O(n)	O(n)	scan right to left with a monotone decreasing stack; pop to track the largest value that is still smaller than some element to its right (the "2"). If a later element is below that "2", a 132 pattern exists.
506	Relative Ranks	Assign ranks ("Gold Medal", "Silver Medal", "Bronze Medal", then 4, 5, ...) to athletes by score.	a max-heap of (score, index) pops athletes in descending score order, so each pop assigns the next rank back to its original position.	O(n log n)	O(n)	Standard
636	Exclusive Time of Functions	Given a log of function start/end times, compute exclusive time per function.	a call stack mirrors execution; when a new call starts, the currently running function pauses (accumulate its slice), and when a call ends, the resumed parent restarts its clock.	O(L) (L = number of logs)	O(n)	Standard
678	Valid Parenthesis String	Check if a string with `(`, `)`, `*` (wildcard) can be a valid parenthesis string.		O(n)	O(1)	track a range `[low, high]` of possible open-paren counts; `*` widens the range (could be `(`, `)`, or empty). Valid iff the range can return to 0 and never goes negative.
703	Kth Largest Element in a Stream	Design a class to find the kth largest element in a stream.	a min-heap capped at size k always holds the k largest seen; its top is the running kth largest.	O(log k) per add	O(k)	Standard
716	Max Stack	Design a stack with push, pop, top, getMax, and popMax operations.		O(n) popMax, O(1) others	O(n)	mirror an auxiliary `maxStack` (like #155). `popMax` pops down to the max into a temp buffer, removes it, then pushes the buffer back — re-establishing both stacks.
735	Asteroid Collision	Simulate asteroids moving in a row: right-moving collide with left-moving.	a stack holds surviving asteroids; a left-moving asteroid only collides with a right-moving one on top, resolving collisions repeatedly until it survives, explodes, or the stack clears.	O(n)	O(n)	Standard
759	Employee Free Time	Find the free time intervals common to all employees given their schedules.	flatten all intervals, sort by start, then sweep tracking the furthest end seen so far; any gap between that end and the next start is common free time.	O(n log n)	O(n)	Standard
844	Backspace String Compare	Check if two strings are equal after processing `#` as backspace.	build each string with a stack where `#` pops the last character, then compare the resulting stacks.	O(n)	O(n)	Standard
856	Score of Parentheses	Calculate the score of a valid parentheses string (empty=0, AB=A+B, (A)=2A or 1 if empty).		O(n)	O(n)	stack holds the accumulated score at each depth; push 0 on `(`, and on `)` collapse the inner score (`max(2*inner, 1)`) into the parent frame.
862	Shortest Subarray with Sum at Least K	Find the length of the shortest subarray with sum at least k (k can be negative).		O(n)	O(n)	monotone increasing deque over prefix sums — pop from the front when a window qualifies, and from the back when a newer prefix is smaller (dominating older larger ones).
907	Sum of Subarray Minimums	Find the sum of subarray minimums for all subarrays.		O(n)	O(n)	monotone increasing stack — for each element as the minimum, count subarrays via distance to the previous strictly-smaller and next smaller-or-equal element, then weight by the value.
921	Minimum Add to Make Parentheses Valid	Find the minimum additions to make a parentheses string valid.	track open parentheses as a counter (stack of size); each unmatched `)` needs an added `(`, and leftover `(` each need a `)`.	O(n)	O(1)	Standard
946	Validate Stack Sequences	Check if a sequence can be produced by a series of push-pop operations on a stack.	simulate — push each value, then greedily pop whenever the stack top matches the next expected popped value. If everything pops, the sequence is valid.	O(n)	O(n)	Standard
973	K Closest Points to Origin	Find the k points closest to the origin.	a max-heap of size k keyed on squared distance keeps the k closest seen; evict the farthest whenever the heap overflows.	O(n log k)	O(k)	Standard
1086	High Five	For each student, compute the average of their top five scores.	keep a min-heap of size 5 per student so only their five highest scores remain, then average those.	O(n log 5)	O(n)	Standard
1190	Reverse Substrings Between	Reverse the substrings between each pair of parentheses (innermost first).		O(n^2)	O(n)	stack of builders — push current builder on `(`; on `)` reverse the inner builder and append it to the parent frame.

#	Name	Description	Intuition	Time	Space	Key Implementation
	Each Pair of Parentheses					
1209	Remove All Adjacent Duplicates in String II	Remove all adjacent duplicates of length k in a string repeatedly.		O(n)	O(n)	stack of (char, count) pairs — increment the top's count on a repeat, and pop the frame once its count reaches k.
1249	Minimum Remove to Make Valid Parentheses	Remove the minimum number of parentheses to make the string valid.		O(n)	O(n)	stack stores indices of unmatched `(`; a `)` with no match is marked for removal, and any `(` left on the stack at the end is also removed.
1337	The K Weakest Rows in a Matrix	Find the k weakest rows in a matrix (fewest soldiers, ties broken by row index).		O(m·n + m log k)	O(k)	max-heap of size k keyed on (soldier count, row index) so the strongest qualifying row sits on top for eviction; the remaining k are the weakest.
1541	Minimum Insertions to Balance a Parentheses String	Find the minimum insertions to make a string balanced (every `(` needs two `)`).		O(n)	O(1)	counter-style stack tracking open `(`; each `(` demands two `)`. Handle `)` carefully since they come in pairs, inserting a `)` when only one is available.
1614	Maximum Nesting Depth of the Parentheses	Find the maximum nesting depth of parentheses in a string.	the stack depth equals the current nesting level; track a running counter for `(` / `)` and record its peak.	O(n)	O(1)	Standard
1792	Maximum Average Pass Ratio	Maximize the average pass ratio by adding extra students optimally.		O((n + extra) log n)	O(n)	max-heap keyed on the marginal gain from adding one student to a class; greedily assign each extra student where it helps most, then push the updated gain back.
1944	Number of Visible People in a Queue	Count the number of people each person can see in a queue (taller blocks view).		O(n)	O(n)	monotone decreasing stack scanned right to left; pop shorter people (each is visible) until a taller one blocks the view — that taller person is visible too.
1985	Find the Kth Largest Integer in the Array	Find the kth largest integer in an array of number strings.		O(n log k)	O(k)	min-heap of size k comparing the numeric strings by length first, then lexicographically (so big-integer values compare correctly without overflow).

BFS (Tree) `bfs.md` (13 problems)

#	Name	Description	Intuition	Time	Space	Key Implementation
102	Binary Tree Level Order Traversal	Return all node values level by level, left to right.		O(n)	O(n)	Standard
107	Binary Tree Level Order Traversal II	Same as #102 but return levels from bottom to top.		O(n)	O(n)	Standard
103	Binary Tree Zigzag Level Order Traversal	Level 1 left-to-right, level 2 right-to-left, alternating.		O(n)	O(n)	Standard
637	Average of Levels in Binary Tree	Return the average value of nodes at each level.		O(n)	O(n)	Standard
199	Binary Tree Right Side View	Return the value of the rightmost node at each level.		O(n)	O(n)	Standard
513	Find Bottom Left Tree Value	Return the leftmost value in the last level (deepest row).		O(n)	O(n)	Standard
515	Find Largest Value in Each Tree Row	Return the maximum value found at each level.		O(n)	O(n)	Standard
1161	Maximum Level Sum of a Binary Tree	Return the smallest level number (1-indexed) whose node sum is maximum.		O(n)	O(n)	Standard
111	Minimum Depth of Binary Tree	Minimum number of nodes from root to the nearest leaf.	BFS reaches nodes in depth order, so the first leaf it dequeues is necessarily the shallowest.	O(n)	O(n)	Standard
116	Populating Next Right Pointers in Each Node	Connect each node's `next` to the node immediately to its right at the same level. Tree is a perfect binary tree.	BFS already visits a level left to right, so just chain each node to the one polled before it.	O(n)	O(n)	Standard
117	Populating Next Right Pointers in Each Node II	Same as #116 but the tree can be any binary tree (not necessarily perfect).		O(n)	O(n)	Standard
993	Cousins in Binary Tree	Two nodes `x` and `y` are cousins if they are at the same depth but have different parents. Return true if `x` and `y` are cousins.	cousins must surface in the *same* BFS level; if only one shows up per level, their depths differ.	O(n)	O(n)	Standard
662	Maximum Width of Binary Tree	Return the maximum width of any level of a binary tree. Width is defined as the length between the leftmost and rightmost non-null nodes, including all the null nodes in between.		O(n)	O(n)	BFS with position tracking. Assign each node a position: root=1, left child of node at pos `p` gets `2*p`, right child gets `2*p+1`. Width of level = `lastPos - firstPos + 1`. Normalize by subtracting `firstPos` of each level to prevent overflow.

DFS / Backtracking `dfs.md` (145 problems)

#	Name	Description	Intuition	Time	Space	Key Implementation
104	Maximum Depth of Binary Tree	Maximum number of nodes along the longest root-to-leaf path.	my depth is just one more than my deeper child's depth.	O(n)	O(h)	Standard
543	Diameter of Binary Tree	Length of the longest path between any two nodes (path does not need to pass through root).	the longest path bends at some node using both arms, but only one arm can extend to its parent.	O(n)	O(h)	Standard
124	Binary Tree Maximum Path Sum	Maximum sum of any path between any two nodes. Values can be negative.	a negative arm is worse than taking nothing, so clamp it to 0 before adding.	O(n)	O(h)	Standard
110	Balanced Binary Tree	Is every node's left and right subtree within 1 height of each other?	fold "is it balanced" into the height return — a poisoned -1 bubbles up and stops the work.	O(n)	O(h)	Standard
226	Invert Binary Tree	Mirror a binary tree — swap left and right subtrees at every node.	invert both children, then swap the pointers at this node.	O(n)	O(h)	Standard
112	Path Sum	Does any root-to-leaf path sum to `targetSum`?	subtract each node from the target on the way down; a leaf with remaining 0 is a hit.	O(n)	O(h)	Standard
113	Path Sum II	Return all root-to-leaf paths that sum to `targetSum`.	the same `path` list is reused for every branch, so undo your add as you unwind.	O(n)	O(n) for output paths	Standard
257	Binary Tree Paths	Return all root-to-leaf paths as strings (`"1->2->5"`).	append to a shared StringBuilder, then truncate back to your entry length to undo.	O(n)	O(n) for output paths	Standard
129	Sum Root to Leaf Numbers	Each root-to-leaf path represents a number (e.g., 1→2→3 = 123). Return the total sum.	build the number digit by digit on the way down (`current*10 + val`); add it up at each leaf.	O(n)	O(h)	Standard
200	Number of Islands	Count the number of islands (connected groups of `'1'`) in a binary grid.	each unvisited land cell starts a new island; flood-fill sinks the whole island so it's counted once.	O(m·n)	O(m·n)	Standard
695	Max Area of Island	Return the area of the largest island.	an island's area is 1 (this cell) plus the areas its four neighbors flood back.	O(m·n)	O(m·n)	Standard
207	Course Schedule	Can you finish all courses given prerequisites? Return false if there is a cycle.	re-entering a node still on the current DFS stack means you looped back to a prerequisite → cycle.	O(V+E)	O(V+E)	Standard
210	Course Schedule II	Return a valid order to take all courses. Return `[]` if a cycle exists.	a node finishes only after all it depends on do, so post-order reversed is a valid prerequisite order.	O(V+E)	O(V+E)	Standard
46	Permutations	All permutations of distinct integers.	order matters, so every position considers all unused elements (no `start` index).	O(n·n!)	O(n)	Standard
78	Subsets	All subsets (power set) of distinct integers.	every prefix on the way down is itself a valid subset, so record at every node.	O(n·2ⁿ)	O(n)	Standard
39	Combination Sum	All combinations summing to target. Same element may be used unlimited times. No duplicate combos.	passing `i` (not `i+1`) lets you pick the same element again; `start` still forbids going backward, so no reordered dups.	O(n · 2ⁿ)	O(target/min)	Standard
40	Combination Sum II	Each element used at most once. Input may have duplicates. No duplicate combos in output.	after sorting, equal values sit together; skipping a repeat at the *same level* prevents producing the same combo twice.	O(n · 2ⁿ)	O(target/min)	Standard
79	Word Search	Given a board of characters, return true if the word exists as a connected path (no cell reused).	the grid itself is the state — temporarily blank a cell so the same path can't reuse it, then restore on the way back.	O(m·n·4^L) where L = word length	O(L)	Standard
131	Palindrome Partitioning	Partition string `s` such that every substring is a palindrome. Return all such partitions.	try every prefix as the first cut; only recurse on the rest when that prefix is a palindrome.	O(n · 2ⁿ)	O(n)	Standard

#	Name	Description	Intuition	Time	Space	Key Implementation
98	Validate Binary Search Tree	Determine if a binary tree is a valid BST. Each node's value must be strictly greater than all values in its left subtree and strictly less than all values in its right subtree.		O(n)	O(h)	Pass min/max bounds down — each call tightens the valid range. Use `long` to handle `Integer.MIN_VALUE` and `Integer.MAX_VALUE` edge cases.
230	Kth Smallest Element in a BST	Find the kth smallest element in a BST (1-indexed).		O(k) average, O(n) worst	O(h)	BST inorder traversal visits nodes in ascending order. Count nodes visited; when count reaches k, record the answer and stop early.
270	Closest Binary Search Tree Value	Given a BST and a `target` (double), return the value in the BST closest to target.		O(h)	O(1)	Use BST structure to navigate in O(h). At each node, compare distance to closest seen so far, then binary-search left or right.
285	Inorder Successor in BST	Given a node `p` in a BST, return its inorder successor (the smallest node with value greater than `p.val`). Return null if no such node exists.		O(h)	O(1)	Navigate the BST. When `root.val > p.val`, this root is a candidate — save it and go left (to find a smaller valid candidate). When `root.val <= p.val`, go right (must find something larger).
687	Longest Univalue Path	Return the length of the longest path in a binary tree where every node along the path has the same value. The path does not need to pass through the root.		O(n)	O(h)	Post-order DFS. At each node, extend the left or right path only if the child's value matches. The path through the current node = leftPath + rightPath. Return the max single-direction extension upward.
37	Sudoku Solver	Fill a partially filled 9×9 Sudoku so every row, column, and 3×3 box contains digits 1-9.		O(9^m) m=empty cells	O(1)	backtracking that tries digits 1-9 in each empty cell, validating against row, column, and box constraints before recursing.
47	Permutations II	Return all unique permutations of an array that may contain duplicates.		O(n·n!)	O(n)	sort first; use a `used[]` array; skip a value if the previous equal value at the same tree level has not been used (prevents duplicate permutations).
51	N-Queens	Place n queens on an n×n board so no two attack each other; return all distinct solutions.		O(n!)	O(n)	backtrack row by row; track occupied columns and both diagonals. Cells on the same `\` diagonal share `r−c`; cells on the same `/` diagonal share `r+c`.
60	Permutation Sequence	Return the kth permutation (1-indexed) of the sequence 1..n.		O(n²)	O(n)	no backtracking — use the factorial number system. Each position's digit is determined directly by `(k−1) / (remaining−1)!`.
90	Subsets II	Return all possible unique subsets of an array that may contain duplicates.		O(n·2ⁿ)	O(n)	sort first; within a recursion level, skip a value equal to its predecessor to avoid duplicate subsets.
216	Combination Sum III	Find all combinations of k numbers from 1-9 (each used at most once) that sum to n.		O(C(9,k))	O(k)	standard combination backtracking with start index `i+1` (no reuse), but with two stopping constraints — count and remaining sum.
549	Binary Tree Longest Consecutive Sequence II	Find the length of the longest consecutive path in a binary tree. The path can be increasing or decreasing and may go through a node (child-parent-child).		O(n)	O(h)	post-order DFS returns a pair `{increasing, decreasing}` for each node. A path through the node combines an increasing run from one child with a decreasing run from the other.
17	Letter Combinations of a Phone Number	Given a string of digits 2-9, return all letter combinations the phone keypad could represent.	backtrack one digit at a time, appending each candidate letter and undoing it.	O(4ⁿ·n)	O(n)	Standard
22	Generate Parentheses	Generate all combinations of n pairs of well-formed parentheses.	add `(` while opens remain; add `)` only while closes outstanding; backtrack each choice.	O(4ⁿ/√n)	O(n)	Standard
36	Valid Sudoku	Determine if a filled-in 9×9 Sudoku board is valid (no repeats in any row, column, or 3×3 box).	encode each digit's row/column/box membership into a HashSet of unique keys; a collision means invalid.	O(1) (fixed 81 cells)	O(1)	Standard
77	Combinations	Return all combinations of k numbers chosen from 1 to n.	standard backtracking with start index advancing to i+1; record when path reaches size k.	O(k·C(n,k))	O(k)	Standard
93	Restore IP Addresses	Return all valid IP addresses formable by inserting dots into a digit string.	backtrack choosing 1-3 digit segments; each segment must be 0-255 and have no leading zero.	O(1) (bounded branching)	O(1)	Standard
94	Binary Tree Inorder Traversal	Return the inorder traversal (left → root → right) of a binary tree.	recurse left, visit node, recurse right.	O(n)	O(h)	Standard

#	Name	Description	Intuition	Time	Space	Key Implementation
95	Unique Binary Search Trees II	Generate all structurally unique BSTs storing values 1..n.	pick each value as root; recursively build all left subtrees from the smaller range and all right subtrees from the larger range, then combine.	O(n·Catalan(n))	O(n·Catalan(n))	Standard
99	Recover Binary Search Tree	Two nodes of a BST were swapped by mistake; recover the tree without changing its structure.	an inorder traversal of a valid BST is ascending; the swapped pair shows up as one or two descents. Track them and swap their values.	O(n)	O(h)	Standard
100	Same Tree	Check if two binary trees are structurally identical with the same node values.	both null is equal; one null or differing values is not; otherwise recurse on both children.	O(n)	O(h)	Standard
101	Symmetric Tree	Check if a binary tree is a mirror image of itself.	compare the outer pairs and inner pairs of two mirrored subtrees.	O(n)	O(h)	Standard
102	Binary Tree Level Order Traversal	Return node values level by level, left to right.	BFS with a queue; drain one level's worth of nodes per outer iteration.	O(n)	O(n)	Standard
103	Binary Tree Zigzag Level Order Traversal	Return node values level by level, alternating left-to-right and right-to-left.	standard BFS, but reverse the level list on alternate levels.	O(n)	O(n)	Standard
105	Construct Binary Tree from Preorder and Inorder Traversal	Reconstruct a binary tree from its preorder and inorder traversals.	the first preorder element is the root; its position in inorder splits left/right subtree sizes.	O(n)	O(n)	Standard
106	Construct Binary Tree from Inorder and Postorder Traversal	Reconstruct a binary tree from its inorder and postorder traversals.	the last postorder element is the root; build right subtree before left since postorder is consumed back to front.	O(n)	O(n)	Standard
107	Binary Tree Level Order Traversal II	Return node values level by level, from bottom to top.	standard BFS, then reverse the list of levels.	O(n)	O(n)	Standard
108	Convert Sorted Array to Binary Search Tree	Convert a sorted array to a height-balanced BST.	pick the middle element as the root so left and right halves are balanced; recurse on each half.	O(n)	O(log n)	Standard
111	Minimum Depth of Binary Tree	Find the shortest path length from root to any leaf.	like max depth, but a node with one missing child must take the existing child's depth (a non-leaf cannot end a root-to-leaf path).	O(n)	O(h)	skip the null side
114	Flatten Binary Tree to Linked List	Flatten a binary tree into a right-pointer linked list following preorder, in place.	reverse-preorder traversal (right, left, node) lets you prepend each node to a running tail.	O(n)	O(h)	Standard
116	Populating Next Right Pointers in Each Node	Connect each node's `next` pointer to its right neighbor in a perfect binary tree, using O(1) extra space.	use already-established `next` links of the current level to thread the next level.	O(n)	O(1)	Standard
117	Populating Next Right Pointers in Each Node II	Same as #116 but for an arbitrary binary tree.	walk each level via `next` links, building the next level's chain through a dummy head since children may be missing.	O(n)	O(1)	Standard
144	Binary Tree Preorder Traversal	Return the preorder traversal (root → left → right) of a binary tree.	visit node, recurse left, recurse right.	O(n)	O(h)	Standard
145	Binary Tree Postorder Traversal	Return the postorder traversal (left → right → root) of a binary tree.	recurse left, recurse right, visit node.	O(n)	O(h)	Standard
199	Binary Tree Right Side View	Return the values visible from the right side, one per level.	BFS each level; the last node dequeued in a level is the rightmost visible one.	O(n)	O(n)	Standard
222	Count Complete Tree Nodes	Count nodes in a complete binary tree faster than O(n).	if leftmost and rightmost depths match, the subtree is perfect (2^h − 1 nodes); otherwise recurse on both children.	$O(\log^2 n)$	O(log n)	perfect subtree shortcut
235	Lowest Common Ancestor of a Binary Search Tree	Find the LCA of two nodes in a BST.	if both values are less than the node, go left; if both greater, go right; otherwise the node is the split point = LCA.	O(h)	O(1)	Standard

#	Name	Description	Intuition	Time	Space	Key Implementation
236	Lowest Common Ancestor of a Binary Tree	Find the LCA of two nodes in a general binary tree.	post-order: if p or q matches the node, return it; the node where both arms return non-null is the LCA.	O(n)	O(h)	Standard
241	Different Ways to Add Parentheses	Compute all results obtainable by parenthesizing an expression differently.	at each operator, split into left and right subexpressions, recursively evaluate both, and combine every pair.	O(Catalan(n))	O(Catalan(n))	Standard
250	Count Univalue Subtrees	Count subtrees in which all nodes share the same value.	post-order; a subtree is univalue if both children are univalue and their values match the node's value.	O(n)	O(h)	Standard
255	Verify Preorder Sequence in Binary Search Tree	Verify whether an array is a valid BST preorder traversal.	use a stack to simulate the path; popping when a value exceeds the top sets a lower bound. Any later value below that bound is invalid.	O(n)	O(n)	Standard
267	Palindrome Permutation II	Return all palindrome permutations of a string.	a palindrome needs at most one odd-count character; build half the palindrome by permuting half the characters, then mirror it.	O((n/2)!)	O(n)	Standard
272	Closest Binary Search Tree Value II	Find the k values in a BST closest to a target.	inorder gives sorted values; keep a sliding window of size k, evicting the farther end while a closer candidate exists.	O(n)	O(n)	values only get farther, stop early
282	Expression Add Operators	Insert +, -, * between digits of a string to reach a target value; return all expressions.	backtrack each operand prefix; track running value and the last multiplied term so * can fold correctly.	$O(4^n)$	O(n)	Standard
291	Word Pattern II	Check if a string follows a pattern where each pattern char maps to a non-empty, non-overlapping substring.	backtrack assigning substrings to pattern characters, enforcing a bijection via two maps.	O(exponential)	O(n)	Standard
293	Flip Game	Given a string of '+' and '-', return all states after flipping one "++" to "--".	scan for each "++" occurrence and produce the flipped string.	$O(n^2)$	O(n)	Standard
294	Flip Game II	Determine if the first player can guarantee a win in Flip Game.	the current player wins if any "++" flip leaves the opponent in a losing position; memoize states.	O(exponential, memoized)	O(states)	Standard
297	Serialize and Deserialize Binary Tree	Serialize a binary tree to a string and deserialize it back.	preorder with explicit "#" null markers uniquely encodes structure; rebuild by consuming tokens in the same order.	O(n)	O(n)	Standard
298	Binary Tree Longest Consecutive Sequence	Find the longest consecutive increasing-by-1 path going from parent to child.	pass the current run length down; extend it when the child is exactly parent + 1, otherwise reset to 1.	O(n)	O(h)	Standard
301	Remove Invalid Parentheses	Remove the minimum number of parentheses to make a string valid; return all distinct results.	BFS by removal count; the first level whose strings include valid ones is the minimum-removal answer.	$O(2^n)$	$O(2^n)$	Standard
306	Additive Number	Check if a string forms an additive sequence (each number is the sum of the two preceding).	backtrack the first two numbers; the rest of the string is forced, so just verify it matches the running sums.	$O(n^3)$	O(n)	Standard
314	Binary Tree Vertical Order Traversal	Return node values ordered by column, then by row.	BFS tracking each node's column; collect values per column, then read columns left to right.	O(n)	O(n)	Standard
320	Generalized Abbreviation	Return all abbreviations of a word (replace any non-adjacent groups of letters with their counts).	for each character, choose to either abbreviate it (extend a count) or keep it literally; backtrack.	$O(2^n \cdot n)$	O(n)	Standard
339	Nested List Weight Sum	Sum each integer in a nested list weighted by its depth.	DFS carrying the current depth; integers add `value·depth`, lists recurse one level deeper.	O(n)	O(d)	Standard
364	Nested List Weight Sum II	Sum each integer weighted by its inverse depth (deepest integers have weight 1).	weight = (maxDepth − depth + 1). Accumulate sum of values at each level and a running unweighted total per BFS level so deeper values get counted more often.	O(n)	O(n)	shallow values re-counted each level
386	Lexicographical Numbers	Return integers 1..n in lexicographical order.	DFS a 10-ary prefix tree: start at each digit, append 0-9 as long as the number stays ≤ n.	O(n)	O(log n)	Standard

#	Name	Description	Intuition	Time	Space	Key Implementation
404	Sum of Left Leaves	Sum the values of all left leaves in a binary tree.	pass down whether a node is a left child; add its value when it is both a left child and a leaf.	O(n)	O(h)	Standard
426	Convert Binary Search Tree to Sorted Doubly Linked List	Convert a BST into a sorted circular doubly linked list in place.	inorder traversal yields sorted order; link each node to the previous one, then close the ring between head and tail.	O(n)	O(h)	Standard
429	N-ary Tree Level Order Traversal	Return the level order traversal of an N-ary tree.	BFS, enqueuing all children of each node per level.	O(n)	O(n)	Standard
437	Path Sum III	Count downward paths (any node to any descendant) summing to a target.	prefix-sum the running root-to-node sum; the number of paths ending here equals how many earlier prefixes equal `current − target`.	O(n)	O(n)	prefix-sum count
440	K-th Smallest in Lexicographical Order	Find the kth smallest integer in 1..n by lexicographical order.	count how many numbers share a prefix; skip whole subtrees of the 10-ary prefix tree when k exceeds their size, otherwise descend.	$O(\log^2 n)$	O(1)	Standard
449	Serialize and Deserialize BST	Serialize and deserialize a BST compactly.	preorder alone reconstructs a BST using value bounds, so no null markers are needed.	O(n)	O(n)	Standard
450	Delete Node in a BST	Delete a key from a BST and return the new root.	find the node; if it has two children, replace its value with the inorder successor (smallest in right subtree) and delete that successor.	O(h)	O(h)	Standard
473	Matchsticks to Square	Determine if matchsticks can form a square (4 equal sides) using all sticks.	backtrack placing each stick into one of 4 side buckets; prune when total isn't divisible by 4 or a stick exceeds the side length.	$O(4^n)$	O(n)	skip equal buckets
488	Zuma Game	Find the minimum balls from hand to insert to clear the board (or -1).	DFS each board state, trying every hand ball at every insertion point, removing groups of 3+; memoize visited states.	O(exponential)	O(states)	Standard
489	Robot Room Cleaner	Clean an entire room with a robot that has only relative move/turn/clean APIs and no map.	DFS with backtracking using relative coordinates; clean, try all four directions, and reverse-move to return after each branch.	O(cells)	O(cells)	Standard
491	Increasing Subsequences	Return all increasing subsequences of length ≥ 2 (input may have duplicates).	backtrack choosing elements ≥ the last picked; within one recursion level use a set to skip duplicate starts. Cannot sort (would destroy original order).	$O(2^n \cdot n)$	O(n)	skip dup at same level
501	Find Mode in Binary Search Tree	Find the mode(s) (most frequent values) in a BST that may have duplicates.	inorder gives sorted values, so equal values are adjacent; track current run count and update the mode list when counts tie or exceed the max.	O(n)	O(h)	Standard
508	Most Frequent Subtree Sum	Find the subtree sum(s) that occur most frequently.	post-order compute each subtree's sum, tally frequencies in a map, then collect the keys with the max frequency.	O(n)	O(n)	Standard
510	Inorder Successor in BST II	Find the inorder successor of a node given only a parent pointer (no root).	if a right subtree exists, the successor is its leftmost node; otherwise climb up until you come from a left child.	O(h)	O(1)	Standard
513	Find Bottom Left Tree Value	Find the leftmost value in the last (deepest) row of a binary tree.	BFS scanning right-to-left so the final node dequeued is the bottom-left value.	O(n)	O(n)	Standard
515	Find Largest Value in Each Tree Row	Return the maximum value in each row of a binary tree.	BFS level by level, tracking the max per level.	O(n)	O(n)	Standard
526	Beautiful Arrangement	Count permutations of 1..n where each number divides or is divisible by its position.	backtrack placing numbers position by position, only choosing a number when it satisfies the divisibility rule.	O(k) where k = valid arrangements	O(n)	Standard
536	Construct Binary Tree from String	Build a binary tree from a string like `4(2(3)(1))(6(5))`.	parse the leading number as the node, then the first parenthesized group as the left subtree and the second as the right subtree.	O(n)	O(h)	Standard

#	Name	Description	Intuition	Time	Space	Key Implementation
538	Convert BST to Greater Tree	Replace each node's value with the sum of all values greater than or equal to it.	reverse inorder (right → node → left) visits values in descending order; carry a running sum and add it into each node.	O(n)	O(h)	Standard
545	Boundary of Binary Tree	Return the boundary: root, left boundary top-down, leaves left-to-right, right boundary bottom-up, no duplicates.	collect three parts separately — left boundary (excluding leaves), all leaves, and right boundary reversed (excluding leaves).	O(n)	O(h)	Standard
559	Maximum Depth of N-ary Tree	Find the maximum depth of an N-ary tree.	depth is 1 plus the max depth among all children.	O(n)	O(h)	Standard
563	Binary Tree Tilt	Sum the tilt of every node, where tilt = |left subtree sum − right subtree sum|.	post-order return each subtree sum; accumulate the absolute difference into a global total.	O(n)	O(h)	Standard
572	Subtree of Another Tree	Check whether `subRoot` is a subtree of `root`.	at every node of `root`, test for structural equality with `subRoot`.	O(m·n)	O(h)	Standard
589	N-ary Tree Preorder Traversal	Return the preorder traversal of an N-ary tree.	visit the node, then recurse into each child in order.	O(n)	O(h)	Standard
590	N-ary Tree Postorder Traversal	Return the postorder traversal of an N-ary tree.	recurse into all children first, then visit the node.	O(n)	O(h)	Standard
606	Construct String from Binary Tree	Build a preorder string with parentheses, e.g. `1(2(4))(3)`.	append the value, then each non-null child wrapped in parentheses; keep empty `()` for a left child only when a right child exists.	O(n)	O(h)	Standard
617	Merge Two Binary Trees	Merge two binary trees by summing overlapping node values.	if either node is null, return the other; otherwise sum values and recurse on both child pairs.	O(n)	O(h)	Standard
637	Average of Levels in Binary Tree	Return the average value of nodes on each level.	BFS each level, summing values and dividing by the level size.	O(n)	O(n)	Standard
652	Find Duplicate Subtrees	Return one root per group of structurally identical, duplicated subtrees.	serialize each subtree post-order into a canonical string; the second time a serialization appears, record its root.	$O(n^2)$	$O(n^2)$	Standard
653	Two Sum IV - Input is a BST	Find whether two nodes in a BST sum to a target.	traverse, storing seen values in a set; for each node check if `target − val` was seen.	O(n)	O(n)	Standard
654	Maximum Binary Tree	Build a tree where the root is the array max, left subtree from the left subarray, right subtree from the right subarray.	find the max in the range as root, recurse on the two halves.	$O(n^2)$	O(n)	Standard
662	Maximum Width of Binary Tree	Return the maximum width of any level, where width counts the gap between the leftmost and rightmost non-null nodes.	assign heap-style indices (left = 2i, right = 2i+1); per level the width is last index − first index + 1.	O(n)	O(n)	Standard
671	Second Minimum Node In a Binary Tree	In a tree where each node's value is the min of its children, find the second smallest value overall.	the root is the global minimum; DFS for the smallest value strictly greater than the root.	O(n)	O(h)	Standard
679	24 Game	Determine if four numbers can be combined with +, −, ×, ÷ to make 24.	backtrack: pick any two numbers, apply each operator, replace them with the result, and recurse until one number remains.	O(1) (bounded)	O(1)	Standard
698	Partition to K Equal Sum Subsets	Determine if an array can be split into k subsets of equal sum.	target = total/k; backtrack filling one bucket at a time, sorting descending to fail fast, skipping when total isn't divisible.	$O(k \cdot 2^n)$	O(n)	Standard
700	Search in a Binary Search Tree	Find the subtree rooted at the node with a given value in a BST.	binary-search the tree: go left if target is smaller, right if larger.	O(h)	O(1)	Standard
701	Insert into a Binary Search Tree	Insert a value into a BST and return the root.	descend like a search until reaching a null child, then attach a new node there.	O(h)	O(h)	Standard
742	Closest Leaf in a Binary Tree	Find the value of the leaf nearest to the node with a given value.	convert the tree to an undirected graph, then BFS from the target node until the first leaf is reached.	O(n)	O(n)	Standard

#	Name	Description	Intuition	Time	Space	Key Implementation
776	Split BST	Split a BST into two trees: one with values ≤ V and one with values > V.	recurse; at each node decide which result tree it belongs to based on V, then reattach the recursively-split child.	O(h)	O(h)	Standard
784	Letter Case Permutation	Return all strings formable by toggling the case of letters in a string.	backtrack character by character; digits have one choice, letters branch into lower and upper case.	$O(2^n \cdot n)$	O(n)	Standard
814	Binary Tree Pruning	Remove every subtree that contains no node with value 1.	post-order prune children first; drop a node if both children become null and its own value is 0.	O(n)	O(h)	Standard
842	Split Array into Fibonacci Sequence	Split a digit string into a Fibonacci-like sequence (each term = sum of previous two), with terms fitting in a 32-bit int.	backtrack the first two numbers; the rest is forced. Prune leading zeros and overflow.	$O(n^2)$	O(n)	Standard
863	All Nodes Distance K in Binary Tree	Find all node values exactly distance k from a target node.	build parent links so the tree becomes an undirected graph, then BFS k levels from the target.	O(n)	O(n)	Standard
865	Smallest Subtree with all the Deepest Nodes	Find the smallest subtree containing all of the tree's deepest leaves.	post-order return (depth, lca). If left and right depths tie, the current node is the LCA; otherwise carry up the deeper side.	O(n)	O(h)	tie → this node is LCA
889	Construct Binary Tree from Preorder and Postorder Traversal	Reconstruct a binary tree from preorder and postorder traversals (any valid tree).	preorder[start] is the root; preorder[start+1] is the left subtree root, whose position in postorder gives the left subtree size.	O(n)	O(h)	Standard
897	Increasing Order Search Tree	Rearrange a BST into a right-skewed tree following inorder order.	inorder traversal; relink each node as the right child of the previous, clearing left pointers.	O(n)	O(h)	Standard
919	Complete Binary Tree Inserter	Design a structure that supports O(1) insertion into a complete binary tree.	keep a BFS queue of nodes that still have a free child slot; insert into the front node and enqueue the new node.	O(1) insert	O(n)	Standard
932	Beautiful Array	Construct an array of 1..n where no three indices i<j<k satisfy 2·a[j] = a[i] + a[k].	divide and conquer: a beautiful array of odds followed by evens stays beautiful, since odd + even is never twice an integer.	O(n log n)	O(n)	Standard
938	Range Sum of BST	Sum all node values within the inclusive range [low, high].	prune branches outside the range using BST ordering.	O(n)	O(h)	Standard
951	Flip Equivalent Binary Trees	Check if two trees are flip-equivalent (can be made identical by swapping some children).	match children either directly or swapped at each node.	O(n)	O(h)	Standard
958	Check Completeness of a Binary Tree	Determine if a binary tree is complete.	BFS including null children; once a null is seen, no real node may follow.	O(n)	O(n)	real node after a gap
987	Vertical Order Traversal of a Binary Tree	Group node values by column, then by row, then by value within the same position.	DFS recording (col, row, val); sort by column, then row, then value.	O(n log n)	O(n)	Standard
988	Smallest String Starting From Leaf	Return the lexicographically smallest root-to-leaf string, where each node maps to a letter 'a'..'z' and the string is read leaf to root.	build the path down, reverse it at each leaf, and track the minimum.	O(n·h)	O(h)	Standard
993	Cousins in Binary Tree	Check if two values are cousins (same depth, different parents).	BFS recording each value's depth and parent; cousins share depth but differ in parent.	O(n)	O(n)	Standard
998	Maximum Binary Tree II	Insert a value into a maximum tree built by appending the value to the original array's end.	since the value was appended last, it lies on the rightmost spine; insert where it first exceeds the current node.	O(h)	O(h)	Standard
1008	Construct Binary Search Tree from Preorder Traversal	Build a BST from its preorder traversal.	the first value is the root; values less than it form the left subtree, the rest form the right. Use an upper bound to decide where to stop.	O(n)	O(h)	Standard
1026	Maximum Difference Between Node and Ancestor	Find the maximum |a − d| over all ancestor a and descendant d pairs.	pass the min and max seen on the path down; at each node update the answer with the largest gap against those extremes.	O(n)	O(h)	Standard

#	Name	Description	Intuition	Time	Space	Key Implementation
1038	Binary Search Tree to Greater Sum Tree	Replace each node's value with the sum of all values greater than or equal to it.	reverse inorder (right → node → left) processes descending order; carry a running sum.	O(n)	O(h)	Standard
1087	Brace Expansion	Expand a string with brace options like `{a,b}c{d,e}` into all sorted concatenations.	parse each position into a sorted list of choices, then backtrack the Cartesian product.	O(product of group sizes)	O(n)	Standard
1104	Path In Zigzag Labelled Binary Tree	Return the root-to-node path of labels in a zigzag-labeled infinite binary tree.	the normal parent of label is label/2, but rows alternate direction, so mirror the parent within its row range.	O(log n)	O(log n)	mirror then halve
1110	Delete Nodes And Return Forest	Delete the given values and return the roots of the resulting forest.	post-order; if a node is deleted, its surviving children become new roots. A node is a root if its parent was deleted (or it's the original root) and it itself survives.	O(n)	O(h)	Standard
1123	Lowest Common Ancestor of Deepest Leaves	Find the LCA of all the deepest leaves.	post-order return (depth, lca); when both arms reach equal depth, the node is the LCA, else carry up the deeper arm.	O(n)	O(h)	Standard
1161	Maximum Level Sum of a Binary Tree	Return the 1-indexed level with the largest sum of node values.	BFS level by level, tracking the level whose sum is maximum.	O(n)	O(n)	Standard
1214	Two Sum BSTs	Given two BSTs and a target, decide if a value from each sums to the target.	store all values of the first BST in a set, then traverse the second checking for the complement.	O(m + n)	O(m)	Standard
1305	All Elements in Two Binary Search Trees	Return all elements from two BSTs in sorted order.	inorder each tree into a sorted list, then merge the two lists like merge sort.	O(m + n)	O(m + n)	Standard
1361	Validate Binary Tree Nodes	Given child arrays, verify the nodes form exactly one valid binary tree.	exactly one node has no parent (the root); every other node has exactly one parent, there are no cycles, and all nodes are reachable from the root.	O(n)	O(n)	Standard
1382	Balance a Binary Search Tree	Rebuild a BST so it becomes height-balanced.	inorder gives a sorted array; build a balanced BST by recursively choosing the middle element as root.	O(n)	O(n)	Standard
1522	Diameter of N-Ary Tree	Find the diameter (longest path between any two nodes) of an N-ary tree.	like binary-tree diameter, but track the two deepest child heights to form the longest path through each node.	O(n)	O(h)	Standard
1644	Lowest Common Ancestor of a Binary Tree II	Find the LCA of two nodes that may not both exist in the tree; return null if either is missing.	standard LCA but count how many of p, q were actually found, so a node returned without seeing both isn't accepted.	O(n)	O(h)	Standard
1650	Lowest Common Ancestor of a Binary Tree III	Find the LCA of two nodes given parent pointers, without root access.	like finding the intersection of two linked lists — walk both pointers up, switching to the other node when one reaches the top; they meet at the LCA.	O(h)	O(1)	Standard

Graph `graph.md` (57 problems)

#	Name	Description	Intuition	Time	Space	Key Implementation
994	Rotting Oranges	Grid of `0` (empty), `1` (fresh), `2` (rotten). Each minute rotten oranges infect adjacent fresh ones. Return minutes until no fresh remain, or -1.		O(V+E)	O(V)	Standard
542	01 Matrix	For each cell, return the distance to the nearest `0`.		O(m·n)	O(m·n)	Standard
127	Word Ladder	Minimum number of transformations from `beginWord` to `endWord`, changing one letter at a time. Each intermediate word must be in `wordList`.		O(N·L·26) where N = words, L = word length	O(N·L)	Standard
130	Surrounded Regions	Flip all `'O'` regions not connected to the border to `'X'`.		O(m·n)	O(m·n)	Standard
417	Pacific Atlantic Water Flow	Return all cells from which water can flow to both the Pacific (top/left border) and Atlantic (bottom/right border).		O(m·n)	O(m·n)	Standard
207	Course Schedule	Can you finish all courses given prerequisites? Detect if there is a directed cycle.		O(V+E)	O(V+E)	Standard
210	Course Schedule II	Return a valid order to take all courses. Return `[]` if impossible.		O(V+E)	O(V+E)	Standard
310	Minimum Height Trees	Find all roots that minimize tree height. At most 2 answers — they are the center node(s) of the longest path.	the best root sits at the center of the longest path; peeling leaves layer by layer converges there.	O(V+E)	O(V+E)	Standard
547	Number of Provinces	Count the number of connected components (provinces) in an undirected graph given as an adjacency matrix.	start with `n` separate sets; every successful merge drops the count by one.	O(n^2·α(n))	O(n)	Standard
684	Redundant Connection	Find the edge that creates a cycle in an undirected graph that would otherwise be a tree.	if both endpoints are already in the same set, this edge closes a cycle — it's the redundant one.	O(n·α(n))	O(n)	Standard
1319	Number of Operations to Make Network Connected	Minimum cable moves to connect all `n` computers. Return -1 if impossible.	each redundant cable can be moved to bridge one gap, so `components − 1` moves suffice if you have enough spare cables.	O(E·α(n))	O(n)	Standard
743	Network Delay Time	Time for a signal from `k` to reach all `n` nodes. Return -1 if unreachable.	every node is reached by its shortest delay; the network is "done" when the slowest of those arrives.	O((V+E)logV)	O(V+E)	Standard
787	Cheapest Flights Within K Stops	Cheapest price from `src` to `dst` using at most `k` stops (k+1 edges).	each round lets paths grow by one more edge, so `k+1` rounds caps the path at `k` stops.	O(k·E)	O(V)	Standard
1514	Path with Maximum Probability	Find the path from `start` to `end` with maximum success probability (product of edge probabilities).	probabilities in [0,1] only shrink when multiplied, so the "closest" (highest-probability) node settles first — Dijkstra still applies.	O((V+E)logV)	O(V+E)	Standard
785	Is Graph Bipartite?	Can graph nodes be split into two groups where every edge connects nodes in different groups?	paint each neighbor the opposite color; a clash means an odd-length cycle, which can't be 2-colored.	O(V+E)	O(V)	Standard
1584	Min Cost to Connect All Points	Connect all points (on a 2D plane) with minimum total Manhattan distance. Return that total cost.	grow one tree, always swallowing the cheapest edge that reaches a node not yet in it.	O(n^2logn)	O(n^2)	Standard
1631	Path With Minimum Effort	Find a path from top-left `[0,0]` to bottom-right `[m−1,n−1]` that minimizes the maximum absolute difference between adjacent cells.		O(m·n·log(m·n))	O(m·n)	Modified Dijkstra. Instead of summing edge weights, the "cost" of a path is the maximum edge cost. Priority queue ordered by effort; `effort[r][c]` = minimum over all paths of their max-abs-diff.

#	Name	Description	Intuition	Time	Space	Key Implementation
269	Alien Dictionary	Given a sorted list of alien words, determine the order of characters in the alien alphabet. Return the order as a string, or empty string if invalid.		O(C) where C = total characters across all words	O(1) (at most 26 nodes)	Build a directed graph of character ordering. Compare each adjacent pair of words to find the first differing character → that gives an edge. Then topological sort. Return empty if there's a cycle or if shorter word is a prefix of longer word in wrong order.
261	Graph Valid Tree	Given `n` nodes labeled 0..n-1 and a list of undirected edges, determine if they form a valid tree (connected, no cycles).		O(n·α(n)) ≈ O(n)	O(n)	A valid tree has exactly `n−1` edges. Use Union Find to detect cycles. If any edge connects two nodes already in the same component → cycle → not a tree.
323	Number of Connected Components in an Undirected Graph	Given `n` nodes and a list of undirected edges, return the number of connected components.		O(n·α(n)) ≈ O(n)	O(n)	decrement on merge
286	Walls and Gates	Each cell is a gate (0), wall (-1), or empty (INF = 2147483647). Fill each empty room with distance to its nearest gate.		O(m·n)	O(m·n)	multi-source BFS — seed the queue with every gate at distance 0, then expand outward simultaneously.
721	Accounts Merge	Each account has a name and a list of emails. Merge accounts that share any email. Return merged accounts with emails sorted.		O(n·α) with sorting O(n log n)	O(n)	Union Find over emails. Assign each unique email an integer id, union all emails within the same account, then group emails by their root.
827	Making A Large Island	In a binary grid you may change at most one 0 to 1. Return the size of the largest island possible.		O(m·n)	O(m·n)	two passes. First DFS-color each island with a unique id (starting at 2) and record its size. Then for every 0, sum the sizes of distinct neighboring islands + 1.
909	Snakes and Ladders	On an n×n board numbered in boustrophedon (zigzag) order, find the minimum dice moves (1-6) from square 1 to square n^2. Ladders/snakes teleport you.		$O(n^2)$	$O(n^2)$	BFS over square numbers. Convert a square number to (row, col) accounting for the zigzag layout.
1091	Shortest Path in Binary Matrix	In an n×n binary grid, find the length of the shortest clear path (cells of 0) from top-left to bottom-right, moving in 8 directions.		O(m·n)	O(m·n)	standard BFS but with 8-directional movement (includes diagonals).
126	Word Ladder II	Find all shortest transformation sequences from `beginWord` to `endWord`, changing one letter at a time, each intermediate word in `wordList`.	BFS level-by-level builds a parent (predecessor) map of which words lead to which; once `endWord` is reached, DFS backtracks through parents to reconstruct every shortest path.	O(N·L·26) BFS + paths	O(N·L)	Standard
133	Clone Graph	Deep-copy an undirected graph where each node has a value and a list of neighbors.	Use a map from original node to its clone to avoid re-cloning and to wire neighbors correctly; BFS the graph, creating clones on first sight.	O(V+E)	O(V)	Standard
277	Find the Celebrity	Among `n` people, find the celebrity: known by everyone, knows nobody. `knows(a, b)` is the API.	A single linear scan narrows to one candidate (if `candidate` knows `i`, the candidate can't be the celebrity, so `i` becomes the new candidate); a second pass verifies.	O(n)	O(1)	Standard
296	Best Meeting Point	Given a grid where `1` marks a home, find the minimum total Manhattan distance to a single meeting point.	Manhattan distance separates into independent x and y components; the optimal coordinate on each axis is the median of the occupied coordinates.	O(m·n)	O(m·n)	Standard
305	Number of Islands II	Land cells are added one at a time to a water grid; after each addition return the current number of islands.		O(k·α(m·n))	O(m·n)	Online Union-Find — when a cell becomes land, union it with any already-land 4-neighbors; track component count.
317	Shortest Distance from All Buildings	Find the empty land cell `0` minimizing total walking distance to every building `1` (`2` = obstacle).	BFS from each building, accumulating distances into every reachable empty cell and counting how many buildings reach it; the answer is the minimum total among cells reachable by all buildings.	O(B·m·n) where B = buildings	O(m·n)	Standard
399	Evaluate Division	Given equations `a/b = value`, answer queries `c/d`. Return -1.0 if undeterminable.	Treat variables as nodes and ratios as weighted directed edges; the answer to a query is the product of edge weights along any path from numerator to denominator.	O(Q·(V+E))	O(V+E)	Standard

#	Name	Description	Intuition	Time	Space	Key Implementation
407	Trapping Rain Water II	Given a 2D height map, compute how much water it can trap after raining.		O(m·n·log(m·n))	O(m·n)	Dijkstra-style BFS from the border inward using a min-heap of boundary heights; water level at a cell is the highest of the minimum boundary it must pass — pop the lowest wall first.
463	Island Perimeter	Given a grid with exactly one island, return its perimeter.	Each land cell contributes 4 sides, minus 2 for every shared edge with an adjacent land cell (counted once per pair).	O(m·n)	O(1)	Standard
529	Minesweeper	Reveal a cell on a Minesweeper board. `'M'` mine, `'E'` unrevealed empty, `'B'` revealed blank, digit = adjacent mine count.		O(m·n)	O(m·n)	BFS/DFS flood fill with 8 directions; if the clicked cell is a mine, mark `'X'`; otherwise count adjacent mines, and only recurse when count is 0.
694	Number of Distinct Islands	Count islands that are distinct by shape (translations are the same, rotations are not).		O(m·n)	O(m·n)	DFS recording the path signature (the direction taken at each step plus a backtrack marker); two islands with identical signatures have the same shape.
733	Flood Fill	Starting at `(sr, sc)`, change all connected same-color pixels to `color`.		O(m·n)	O(m·n)	Standard
752	Open the Lock	A 4-wheel lock starts at `"0000"`; each move turns one wheel ±1. Find minimum moves to reach `target`, avoiding `deadends`.		O(10^4 · 8)	O(10^4)	BFS over the 4-digit state space; each state has 8 neighbors (each wheel up or down). Treat deadends as visited.
778	Swim in Rising Water	At time `t` you can stand on any cell with elevation ≤ `t`. Find the least time to swim from `(0,0)` to `(n-1,n-1)`.		O(n²·log(n))	O(n²)	Modified Dijkstra — minimize the maximum elevation along the path (not the sum). Min-heap ordered by current max elevation.
797	All Paths From Source to Target	In a DAG given as adjacency list `graph`, return all paths from node `0` to node `n-1`.	Since it is a DAG with no cycles, plain DFS backtracking enumerates every path; no visited set is needed.	O(2^V · V)	O(V)	Standard
802	Find Eventual Safe States	A node is safe if every path from it eventually reaches a terminal node (no outgoing edges). Return all safe nodes sorted.		O(V+E)	O(V+E)	Topological sort on the reversed graph (Kahn's) — start from terminal nodes (out-degree 0); a node becomes safe once all its outgoing edges lead to safe nodes.
815	Bus Routes	`routes[i]` is the stops served by bus `i`. Find the minimum number of buses to travel from `source` to `target`.		O(sum of route lengths)	O(sum of route lengths)	BFS where the "nodes" are buses (routes); build stop→routes index, then BFS route-to-route through shared stops, counting buses taken.
847	Shortest Path Visiting All Nodes	Find the length of the shortest path in an undirected graph that visits every node (may revisit nodes/edges).		O(2^n · n²)	O(2^n · n)	BFS over states `(node, visitedMask)` where `mask` is a bitmask of visited nodes; the goal is any state with all bits set. Start BFS from every node simultaneously.
851	Loud and Rich	`richer[i] = [a, b]` means `a` is richer than `b`; `quiet[i]` is the quietness of person `i`. For each person find the quietest person among everyone at least as rich (themselves included).		O(V+E)	O(V+E)	Topological sort (Kahn's) on the richer→poorer graph; propagate the quietest-known answer from richer people down to poorer ones in topo order.
886	Possible Bipartition	Given `dislikes` pairs, split `n` people into two groups so disliking people are never in the same group.		O(V+E)	O(V+E)	Standard
913	Cat and Mouse	A mouse (start node 1) and cat (start node 2) move on a graph; mouse wins by reaching node 0, cat wins by catching the mouse, else draw. Return 0/1/2 with optimal play.		O(V³)	O(V²)	Game-theory BFS over states `(mouse, cat, turn)`; start from known terminal states and propagate results backward (retrograde analysis) by counting undecided moves.
934	Shortest Bridge	A grid has exactly two islands of `1`s. Find the minimum number of `0`s to flip to connect them.		O(n²)	O(n²)	DFS to mark the first island (seeding a BFS queue with its cells), then multi-source BFS expanding outward until reaching the second island; BFS levels = bridge length.
1034	Coloring A Border	Color the border cells of the connected component containing `(row, col)` with `color`. A border cell touches the grid edge or a cell of a different original color.		O(m·n)	O(m·n)	BFS over same-color connected cells; mark a cell as a border when it has fewer than 4 same-component neighbors, recolor borders after traversal.
111	Path With Maximum	Find a path from top-left to bottom-right maximizing the minimum cell value		O(m·n·log(m·n))	O(m·n)	Modified Dijkstra with a max-heap — the path "score" is the minimum cell value seen; greedily

#	Name	Description	Intuition	Time	Space	Key Implementation
02	Minimum Value	along the path (4-directional).				expand the highest-scoring frontier first.
1136	Parallel Courses	Courses 1..n with prerequisite `relations`; in each semester take all courses whose prerequisites are done. Return minimum semesters, or -1 if impossible.		O(V+E)	O(V+E)	Topological sort (Kahn's) processed level-by-level — each BFS level is one semester; if not all courses are processed, a cycle exists.
1162	As Far from Land as Possible	In a grid of `0` (water) and `1` (land), find the water cell whose distance to the nearest land is maximized; return that distance, or -1.		$O(n^2)$	$O(n^2)$	Multi-source BFS seeded with all land cells; the last BFS level expanded gives the maximum distance.
1197	Minimum Knight Moves	On an infinite chessboard, find the minimum knight moves from `(0,0)` to `(x, y)`.		$O(\max(x,y)^2)$	$O(\max(x,y)^2)$	BFS over knight moves (8 jump offsets); exploit symmetry by working in the first quadrant to bound the visited set.
1202	Smallest String With Swaps	Given index `pairs` that may be swapped any number of times, return the lexicographically smallest string achievable.		$O(n \cdot \log(n) + E \cdot \alpha(n))$	O(n)	Union-Find groups swappable indices into connected components; within each component, sort the characters and place them back in ascending index order.
1245	Tree Diameter	Given the edges of an undirected tree, return its diameter (the number of edges on the longest path between any two nodes).		O(V+E)	O(V+E)	Two BFS passes — BFS from any node finds the farthest node `u`; a second BFS from `u` gives the diameter as the maximum distance.
1293	Shortest Path in a Grid with Obstacles Elimination	Find the shortest path from `(0,0)` to `(m-1,n-1)` in a grid where you may eliminate at most `k` obstacles (`1`s).		$O(m \cdot n \cdot k)$	$O(m \cdot n)$	BFS over states `(row, col, remainingK)`; the visited set tracks the best remaining eliminations per cell so a cell can be revisited with more budget.
1559	Detect Cycles in 2D Grid	Return true if the grid contains a cycle of length ≥ 4 made of the same character.		$O(m \cdot n \cdot \alpha(m \cdot n))$	$O(m \cdot n)$	Union-Find over grid cells — for each cell union it with its up and left same-character neighbors; if a union finds them already connected, a cycle exists.
1743	Restore the Array From Adjacent Pairs	Given all adjacent pairs of an array (in any order), reconstruct the original array.		O(n)	O(n)	Build an adjacency graph; the two endpoints have a single neighbor. Start from an endpoint and walk the path, avoiding the previous node.

Greedy `greedy.md` (26 problems)

#	Name	Description	Intuition	Time	Space	Key Implementation
435	Non-overlapping Intervals	Remove the minimum number of intervals to make the rest non-overlapping.	Always keep the interval that finishes earliest — it leaves the most room for the rest, so greedily picking by end maximizes how many you keep.	O(n log n)	O(1)	Standard
452	Minimum Number of Arrows to Burst Balloons	Balloons span `[start, end]`. An arrow shot at `x` pops all balloons with `start ≤ x ≤ end`. Minimum arrows to pop all balloons.	Shoot the arrow at the earliest end point to pop every balloon overlapping it; only start a new arrow when a balloon begins past the current arrow.	O(n log n)	O(1)	Standard
646	Maximum Length of Pair Chain	Given pairs `[a, b]`, form a chain where `b < c` for each consecutive pair `[a,b]` → `[c,d]`. Maximum chain length.	Picking the pair that ends earliest each time leaves the most slack for later links, maximizing the chain.	O(n log n)	O(1)	Standard
56	Merge Intervals	Merge all overlapping intervals. Return the minimum set of non-overlapping intervals.	sort to make overlaps adjacent, then sweep once — each interval only ever compares against the most recently kept one.	O(n log n)	O(n)	Standard
57	Insert Interval	Insert a new interval into a sorted list of non-overlapping intervals; merge if necessary. The input is already sorted — no sort needed.	Walk the sorted list in three phases — copy everything that ends before the new interval, absorb everything that overlaps it, then copy the rest.	O(n)	O(n)	Standard
253	Meeting Rooms II	Minimum number of meeting rooms required to host all meetings.	USE WHEN counting simultaneous/overlapping resources — "minimum rooms", "max concurrent". The heap size at any moment is exactly how many meetings are running at once.	O(n log n)	O(n)	Standard
55	Jump Game	Each element is the max jump from that position. Can you reach the last index?	Sweep left to right tracking the farthest index reachable; if you ever stand past that frontier you're stuck.	O(n)	O(1)	Standard
45	Jump Game II	Minimum jumps to reach the last index.	Treat each jump as a BFS layer — extend through the whole current reach before incrementing the jump count.	O(n)	O(1)	Standard
763	Partition Labels	Partition string into as many parts as possible so each letter appears in at most one part. Return the size of each part.	A partition can't close until you pass the last occurrence of every letter seen inside it, so extend `end` to that frontier and cut when you reach it.	O(n)	O(1)	Standard
406	Queue Reconstruction by Height	People given as `[h, k]` (height h, k taller/equal people in front). Reconstruct the queue.	Place tallest first so that inserting a person at index `k` is always correct — only taller/equal people are already placed, and shorter insertions later don't disturb their counts.	$O(n^2)$	O(n)	Standard
252	Meeting Rooms	Given an array of meeting intervals, determine if a person could attend all meetings (no two meetings overlap).	After sorting by start, any overlap must be between adjacent intervals, so one linear scan comparing neighbors is enough.	O(n log n)	O(1)	start < previous end = overlap
134	Gas Station	There are n gas stations on a circular route. `gas[i]` is the gas at station i; `cost[i]` is the gas needed to travel from i to i+1. Return the starting station index if you can complete the circuit, or -1 if not.	If the tank ever drops below 0, no station between the old start and here can work, so jump the start past the failure point; a valid start exists iff total gas ≥ total cost.	O(n)	O(1)	reset start when tank goes negative
135	Candy	Assign minimum candies to children in a line: each child gets at least 1 candy, and a child with a higher rating than an adjacent neighbor gets more candies than that neighbor.	Two passes capture the two directions of the constraint — a left-to-right pass enforces "higher than left neighbor gets more", a right-to-left pass enforces "higher than right neighbor gets more". Taking the max of both satisfies both.	O(n)	O(n)	higher than left neighbor
274	H-Index	Given an array of citation counts, find the largest `h` such that at least `h` papers each have at least `h` citations.	After sorting ascending, if a paper at index `i` has `citations[i]` citations, then there are `n − i` papers with at least that many. The h-index is found where `citations[i] >= n − i` first holds.	O(n log n)	O(1)	n-i papers have >= citations[i]
455	Assign Cookies	Each child `i` has a greed factor `g[i]` (minimum cookie size needed); each cookie `j` has size `s[j]`. Each cookie satisfies at most one child. Maximize the number of content children.	Sort both arrays; give the smallest sufficient cookie to the least greedy child. This never wastes a large cookie on a child a small one could satisfy.	O(n log n)	O(1)	Standard
575	Distribute Candies	Distribute `n` candies (the holder eats exactly `n/2`) to maximize the number of distinct candy types eaten.	You can eat at most `n/2` candies, and at best one of each distinct type. So the answer is the smaller of the number of distinct types and `n/2`.	O(n)	O(n)	capped by half the total
605	Can Place Flowers	Given a flowerbed (array of 0s and 1s where 1 is a planted flower), determine if `n` new flowers can be planted without any two flowers being adjacent.	Greedily plant a flower at the first empty plot whose neighbors are also empty — planting as early as possible never blocks a future placement that a later choice would allow.	O(n)	O(1)	plot and both neighbors empty

#	Name	Description	Intuition	Time	Space	Key Implementation
630	Course Schedule III	Each course has `(duration, lastDay)`. Starting at day 0 and taking courses one at a time, find the maximum number of courses that can be taken so each finishes by its `lastDay`.	Sort by deadline so deadlines are considered in order. Greedily take each course; if total time exceeds the current deadline, drop the previously taken course with the largest duration (a max-heap) — swapping out the longest course frees the most time while keeping the count.	O(n log n)	O(n)	over deadline → drop longest
881	Boats to Save People	Each boat carries at most 2 people with total weight at most `limit`. Given people's weights, find the minimum number of boats to rescue everyone.	Pair the heaviest remaining person with the lightest remaining one if they fit together; otherwise the heaviest goes alone. Sorting plus two pointers makes this pairing optimal.	O(n log n)	O(1)	Standard
986	Interval List Intersections	Given two lists of pairwise-disjoint, sorted intervals, return the intersection of the two lists.	Two pointers walk both sorted lists together. The overlap of the two current intervals is `[max(starts), min(ends)]`, which is valid only when `max(starts) <= min(ends)`. Advance whichever interval ends first.	O(m + n)	O(m + n)	valid overlap
1005	Maximize Sum Of Array After K Negations	Given an array and `k`, perform exactly `k` operations, each negating one element (the same index may be chosen repeatedly), to maximize the array sum.	Flipping the most negative number first yields the biggest gain. After all negatives are flipped (or k runs out), any leftover flips should land on the smallest-magnitude element, since flipping it twice is a no-op.	O(n log n)	O(1)	leftover odd flips hit smallest
1094	Car Pooling	Given trips `[numPassengers, from, to]` and a car capacity, determine whether all trips can be completed without exceeding capacity at any point.	A difference array (sweep line) over locations records passengers boarding at `from` and leaving at `to`. Running the prefix sum gives the occupancy at each point; if it ever exceeds capacity, fail.	O(n + maxLocation)	O(maxLocation)	over capacity → fail
1272	Remove Interval	Given a sorted list of disjoint intervals and an interval `toBeRemoved`, return the set of intervals after removing every point that lies inside `toBeRemoved`.	Each interval is either fully outside the removal range (keep as is) or partially overlaps it (keep the non-overlapping left and/or right slivers). Walk once and emit the surviving pieces.	O(n)	O(n)	keep left sliver
1326	Minimum Number of Taps to Open to Water a Garden	A garden spans `[0, n]`. Tap `i` at position `i` waters `[i - ranges[i], i + ranges[i]]`. Find the minimum number of taps to water the whole garden, or -1 if impossible.	This is a jump-game in disguise: for each starting point record the farthest reach of any tap covering it, then greedily extend coverage layer by layer like minimum jumps.	O(n)	O(n)	gap in coverage → impossible
1353	Maximum Number of Events That Can Be Attended	Each event `[start, end]` can be attended on any single day in `[start, end]`. Attend at most one event per day. Find the maximum number of events you can attend.	Process days in increasing order; among all events available on the current day, attend the one that ends soonest (a min-heap by end day) — saving longer-lived events for later days.	O(n log n)	O(n)	attend earliest-ending event
1488	Avoid Flood in The City	`rains[i] > 0` means lake `rains[i]` fills with rain on day `i`; `rains[i] == 0` is a dry day on which you may dry exactly one lake. A full lake that rains again floods. Return an array where dry days hold the lake to dry (any valid choice) and rain days hold -1, or an empty array if flooding is unavoidable.	When a lake that is already full rains again, you must have used a dry day between its two rains. Greedily assign the earliest available dry day that falls after the lake was last filled — a TreeSet of dry-day indices gives that earliest day.	O(n log n)	O(n)	no dry day available → flood

Dynamic Programming `dynamic_programming.md` (84 problems)

#	Name	Description	Intuition	Time	Space	Key Implementation
70	Climbing Stairs	Each step you can climb 1 or 2 steps. How many distinct ways to reach step n?	the last move was a 1-step or a 2-step, so ways to reach n = ways to n-1 + ways to n-2.	O(n)	O(n)	Standard
198	House Robber	Rob houses on a line — can't rob two adjacent. Maximize total.	at each house, either skip it (keep `dp[i−1]`) or rob it (`dp[i−2] + value`, since i-1 is now off-limits).	O(n)	O(n)	Standard
213	House Robber II	Houses arranged in a circle — first and last are adjacent. Can't rob both.	breaking the circle means either the first house is excluded or the last is — solve both lines and keep the better.	O(n)	O(1)	Standard
300	Longest Increasing Subsequence	Length of the longest strictly increasing subsequence.	the best subsequence ending at i extends the longest earlier one whose last value is still smaller.	$O(n^2)$	O(n)	Standard
139	Word Break	Can `s` be segmented into a sequence of dictionary words?	a prefix is breakable if some earlier breakable cut leaves a dictionary word as the final piece.	$O(n^2 \cdot L)$ where L = substring length	O(n)	Standard
416	Partition Equal Subset Sum	Can the array be partitioned into two subsets with equal sum?		O(n)	O(sum)	Standard
494	Target Sum	Assign `+` or `−` to each number to reach `target`. How many ways?		O(n)	O(sum)	Standard
474	Ones and Zeroes	Given strings of '0'/'1', find the largest subset using at most `m` zeros and `n` ones.		O(l)	O(mn)	Standard
322	Coin Change	Minimum number of coins to make `amount`. Coins can be reused.		O(amount)	O(amount)	Standard
518	Coin Change II	Number of distinct combinations (order doesn't matter) to make `amount`.		O(amount)	O(amount)	Standard
377	Combination Sum IV	Number of ordered sequences (order matters) that sum to `target`.		O(target)	O(target)	Standard
1143	Longest Common Subsequence	Length of the longest subsequence present in both strings.	if the last chars match, they're part of the LCS (+1 on the diagonal); otherwise drop one char from whichever string and keep the better.	O(m·n)	O(m·n)	Standard
72	Edit Distance	Minimum operations (insert, delete, replace) to convert `word1` to `word2`.	matched chars cost nothing (diagonal); otherwise take the cheapest of replace/delete/insert and add 1.	O(m·n)	O(m·n)	Standard
115	Distinct Subsequences	Number of ways `s` contains `t` as a subsequence.	you can always skip the current char of s; if it matches the current char of t, you may also use it to advance t.	O(m·n)	O(m·n)	Standard
647	Palindromic Substrings	Count all palindromic substrings of `s`.	`s[i..j]` is a palindrome when its two ends match AND the inside `s[i+1..j−1]` already is.	$O(n^2)$	$O(n^2)$	Standard
516	Longest Palindromic Subsequence	Length of the longest subsequence of `s` that is a palindrome.	matching ends add 2 to the inner range's best; otherwise drop one end and keep the larger.	$O(n^2)$	$O(n^2)$	Standard
312	Burst Balloons	Burst all balloons to maximize coins. Coins from bursting balloon `k` between open boundaries `i` and `j` = `balloons[i] * balloons[k] * balloons[j]`.		$O(n^3)$	$O(n^2)$	Standard
62	Unique Paths	Number of paths from top-left to bottom-right, moving only right or down.	you reach a cell only from above or from the left, so its path count is the sum of those two.	O(m·n)	O(m·n)	Standard
64	Minimum Path Sum	Minimum sum path from top-left to bottom-right, moving only right or down.	the cheapest way into a cell is its own value plus the cheaper of the cell above or to its left.	O(m·n)	O(m·n)	Standard
22	Maximal Square	Find the largest all-1 square in a binary matrix. Return its area.		O(m·n)	O(m·n)	Standard

#	Name	Description	Intuition	Time	Space	Key Implementation
1						
329	Longest Increasing Path in a Matrix	Find the longest strictly increasing path. Can move in all 4 directions.	strictly increasing values forbid revisiting, so each cell's longest path is 1 + the best of its larger neighbors — cache it.	O(m·n)	O(m·n)	Standard
121	Best Time to Buy and Sell Stock (at most 1 transaction)		`hold` is the best profit if you currently own a share bought at the lowest price seen; sell whenever today's price beats that.	O(n)	O(1)	Standard
122	Best Time to Buy and Sell Stock II (unlimited transactions)		with unlimited trades, buying can fund itself from accumulated `cash`, so capture every upward move.	O(n)	O(1)	Standard
123	Best Time to Buy and Sell Stock III (at most 2 transactions)		the second buy can only spend profit left after the first sell, so chain the states in trade order.	O(n)	O(1)	Standard
309	Best Time to Buy and Sell Stock with Cooldown	After selling, must wait 1 day before buying again.	the cooldown forces buying to draw from cash that's at least one day old, so carry yesterday's cash forward.	O(n)	O(1)	Standard
714	Best Time to Buy and Sell Stock with Transaction Fee	Pay a fee per transaction (on sell). Unlimited transactions.	the fee just shrinks every sale, so a trade is only worth making when the gain clears the fee.	O(n)	O(1)	Standard
746	Min Cost Climbing Stairs	Each step has a cost. You can start from index 0 or 1, and from each step you can climb 1 or 2 steps. Return the minimum cost to reach the top (past the last index).		O(n)	O(n)	Standard
91	Decode Ways	A string of digits can be decoded: '1'-'9' → single letter, "10"-"26" → two-digit letter. Count total decoding ways.		O(n)	O(n)	Two sources at each position. If the current digit is non-zero, it can be decoded alone (`dp[i] += dp[i-1]`). If the previous two digits form a valid two-letter code (10-26), add `dp[i-2]` .
279	Perfect Squares	Return the minimum number of perfect squares (1, 4, 9, 16, ...) that sum to `n` .		O(n√n)	O(n)	iterate all squares ≤ i
96	Unique Binary Search Trees	Return the number of structurally unique BSTs that store values 1 to n.		$O(n^2)$	O(n)	Catalan number recurrence. `dp[i]` = number of unique BSTs with i nodes. When root = j, left subtree has j-1 nodes, right subtree has i-j nodes: `dp[i] += dp[j-1] * dp[i-j]` .
63	Unique Paths II	A robot moves from top-left to bottom-right in an m×n grid with obstacles. Count unique paths. Cells with `obstacleGrid[i][j] == 1` are blocked.		O(m·n)	O(m·n)	Same as #62 (Unique Paths) but skip cells with obstacles: `dp[i][j] = 0` if blocked, else `dp[i-1][j] + dp[i][j-1]` .
583	Delete Operation for Two Strings	Return the minimum number of deletions to make `word1` and `word2` equal.		O(m·n)	O(m·n)	Reduce to LCS. Minimum deletions = `len(word1) + len(word2) - 2 * LCS(word1, word2)` . Compute LCS using standard 2D DP.
368	Largest Divisible Subset	Find the largest subset of distinct positive integers such that for every pair (a, b), either `a % b == 0` or `b % a == 0` .		$O(n^2)$	O(n)	Sort first, then apply LIS-style DP. `dp[i]` = size of largest divisible subset ending at `nums[i]` . If `nums[i] % nums[j] == 0` , then `dp[i] = max(dp[i], dp[j] + 1)` . Track parent indices to reconstruct the subset.
931	Minimum Falling Path Sum	Given an n×n matrix, return the minimum sum of a falling path. A falling path starts at any element in the first row and chooses the element directly below or diagonally adjacent in each row.		$O(n^2)$	$O(n^2)$	Grid DP with three sources from the row above. `dp[i][j] = matrix[i][j] + min(dp[i-1][j-1], dp[i-1][j], dp[i-1][j+1])` .
10	Regular Expression Matching	Implement regex matching with `.` (any single char) and `*` (zero or more of the preceding element). Match must cover the entire input string.		O(m·n)	O(m·n)	2D DP. `dp[i][j]` = whether `s[0..i)` matches `p[0..j)` . The `*` has two cases: zero occurrences (`dp[i][j-2]`) or one-more occurrence when the preceding pattern char matches `s[i-1]` .

#	Name	Description	Intuition	Time	Space	Key Implementation
87	Scramble String	Given s1 and s2, return true if s2 is a scrambled string of s1 (formed by recursively partitioning into two non-empty substrings and optionally swapping them).		$O(n^4)$	$O(n^3)$	memoized recursion over (i1, i2, len). At each length, try every split point both swapped and non-swapped.
1049	Last Stone Weight II	Repeatedly smash the two heaviest stones (result is their difference). Return the smallest possible weight of the remaining stone.		O(n·sum)	O(sum)	equivalent to splitting stones into two groups to minimize the difference of their sums. 0/1 knapsack to find the max subset sum ≤ total/2.
1312	Minimum Insertion Steps to Make a String Palindrome	Return the minimum number of insertions needed to make a string a palindrome.		$O(n^2)$	$O(n^2)$	answer = n − longest palindromic subsequence (LPS). LPS = LCS of the string and its reverse; or solve directly via interval DP.
44	Wildcard Matching	Match string `s` against pattern `p` with `?` (any single char) and `*` (any sequence including empty).	`*` either consumes one more char of `s` (`dp[i−1][j]`) or matches empty (`dp[i][j−1]`); everything else is a 1-to-1 char/ `?` match.	O(m·n)	O(m·n)	'*' matches empty prefix
53	Maximum Subarray	Find the contiguous subarray with the largest sum.	the best sum ending at `i` either extends the previous best or restarts at `nums[i]` (Kadane).	O(n)	O(1)	Kadane restart vs extend
97	Interleaving String	Return whether `s3` is formed by interleaving `s1` and `s2` while preserving each one's order.	the last char of `s3[0..i+j)` comes from either `s1` or `s2` ; reuse the answer for the smaller prefix.	O(m·n)	O(m·n)	take from s1
120	Triangle	Find the minimum path sum from top to bottom, moving to adjacent indices on the row below.	working bottom-up, each cell's best is its value plus the cheaper of the two children directly below.	$O(n^2)$	O(n)	bottom-up over rows
132	Palindrome Partitioning II	Return the minimum number of cuts so that every substring of the partition is a palindrome.	if `s[j..i]` is a palindrome, a cut after `j−1` lets `cuts[i] = cuts[j−1] + 1` ; precompute palindromes with interval DP.	$O(n^2)$	$O(n^2)$	interval DP precompute palindromes
140	Word Break II	Return all sentences obtained by segmenting `s` into space-separated dictionary words.	memoize on each suffix the list of valid segmentations, prepending each matching word to the segmentations of the remainder.	$O(n^2 \cdot 2^n)$ worst case	$O(2^n)$	memoized DFS returns sentences
152	Maximum Product Subarray	Find the contiguous subarray with the largest product.	a negative number flips max and min, so track both; the running max product ending at `i` may come from the prior max or min.	O(n)	O(1)	negative swaps max/min
174	Dungeon Game	Find the minimum starting health so the knight survives from top-left to bottom-right (each cell adds/subtracts health, must stay ≥ 1).	health needed depends on the future, so fill from the bottom-right: needed entering a cell = max(1, min child need − cell value).	O(m·n)	O(m·n)	fill from bottom-right
188	Best Time to Buy and Sell Stock IV	Maximize profit with at most `k` transactions.	for each allowed transaction count `t` , track best buy and sell; this is #123 generalized to `k` chained state pairs.	O(n·k)	O(k)	k chained state pairs
264	Ugly Number II	Find the nth ugly number (positive integers whose only prime factors are 2, 3, 5).	every ugly number is a previous ugly number times 2, 3, or 5; merge the three multiples with pointers.	O(n)	O(n)	merge multiples via pointers
313	Super Ugly Number	Find the nth super ugly number whose prime factors all come from a given `primes` list.	generalize #264 to arbitrary primes — keep one pointer per prime and pick the minimum next candidate.	O(n·k)	O(n+k)	one pointer per prime
337	House Robber III	Houses form a binary tree; you cannot rob two directly-linked houses. Maximize the total.	each node returns two values — best if it is robbed (skip children) vs not robbed (children free to choose).	O(n)	O(h)	tree DP returns {notRob, rob}
343	Integer Break	Break `n` into the sum of at least two positive integers maximizing their product.	for each split `j` , the rest can stay as `i−j` or be broken further (`dp[i−j]`); take the best.	$O(n^2)$	O(n)	split point, break further or not
375	Guess Number Higher or Lower II	Find the minimum amount of money to guarantee a win when guessing a number in `[1, n]` , paying the guessed value on each wrong guess.	for range `[i,j]` , guessing `k` costs `k` plus the worst of the two resulting subranges; minimize over `k` .	$O(n^3)$	$O(n^2)$	interval DP over range length
376	Wiggle Subsequence	Find the length of the longest subsequence whose consecutive differences strictly alternate between positive and negative.	track the best subsequence ending with an up-move and with a down-move; each rise extends `down` , each fall extends `up` .	O(n)	O(1)	rise extends down-ending seq

#	Name	Description	Intuition	Time	Space	Key Implementation
413	Arithmetic Slices	Count the number of arithmetic subarrays (length ≥ 3, constant difference).	if `nums[i]` continues the arithmetic run ending at `i−1`, it adds `dp[i−1]+1` new slices ending at `i`.	O(n)	O(1)	extend arithmetic run
446	Arithmetic Slices II - Subsequence	Count the arithmetic subsequences (length ≥ 3) of the array.	for each pair `(j, i)` with difference `d`, weak slices ending at `i` extend those ending at `j`; promote weak to valid when length reaches 3.	$O(n^2)$	$O(n^2)$	per-index map keyed by difference
486	Predict the Winner	Two players alternately take a number from either end; decide whether player 1 can win (score ≥ player 2).	on range `[i,j]` the current player maximizes value − opponent's best on the remaining range.	$O(n^2)$	$O(n^2)$	interval DP, score difference
509	Fibonacci Number	Return the nth Fibonacci number.	each term is the sum of the previous two — the canonical linear 1D recurrence.	O(n)	O(1)	Standard
552	Student Attendance Record II	Count length-`n` attendance records that are rewardable (at most one 'A', no three consecutive 'L'), modulo 1e9+7.	state = (number of 'A's so far 0/1, trailing 'L' run 0/1/2); each day append P, A, or L respecting limits.	O(n)	O(1)	state = (A count, trailing L run)
576	Out of Boundary Paths	Count paths that move a ball off the grid in exactly `maxMove` steps from a start cell, modulo 1e9+7.	moving off the boundary counts as one path; otherwise spread current-step counts to 4 neighbors for the next step.	$O(\text{maxMove}\cdot m\cdot n)$	$O(m\cdot n)$	spread to 4 neighbors
600	Non-negative Integers without Consecutive Ones	Count integers in `[0, n]` whose binary representation has no two adjacent 1 bits.	scan bits high-to-low; precompute Fibonacci-style counts of valid suffixes, adding them when a free bit is `1`, and stop early if two consecutive 1s appear in `n`.	O(1) (32 bits)	O(1)	fib[i] = valid i-bit strings
629	K Inverse Pairs Array	Count permutations of `1..n` with exactly `k` inverse pairs, modulo 1e9+7.	inserting `n` into a permutation of `1..n−1` adds `0..n−1` inversions; this gives a sliding-window sum over the previous row.	$O(n\cdot k)$	$O(n\cdot k)$	prefix sums of previous row
638	Shopping Offers	Buy exactly `needs[i]` of each item at minimum cost, given unit prices and special bundle offers (each usable multiple times).	treat the remaining needs vector as the state; either buy everything at unit price, or apply an affordable offer and recurse.	$O(\text{offers}\cdot\prod \text{needs}_i)$	$O(\prod \text{needs}_i)$	DFS+memo over needs vector
639	Decode Ways II	Count decodings of a digit string where `*` is a wildcard for digits 1-9, modulo 1e9+7.	extend #91 — each position can decode one char (count 1-9 options) or two chars (count valid 10-26 combinations); `*` multiplies the options.	O(n)	O(n)	single char, count options
650	2 Keys Keyboard	Starting with one 'A', using only Copy-All and Paste, return the minimum operations to obtain `n` 'A's.	the answer is the sum of `n`'s prime factors — building `i` from a divisor `j` costs `i/j` operations.	$O(n\sqrt{n})$	O(n)	build i from divisor j
673	Number of Longest Increasing Subsequence	Count the number of longest strictly increasing subsequences.	alongside LIS length, track how many ways achieve it; a longer extension resets the count, an equal-length extension accumulates it.	$O(n^2)$	O(n)	longer extension resets count
674	Longest Continuous Increasing Subsequence	Find the length of the longest continuous (contiguous) strictly increasing subarray.	extend the current run while values rise, otherwise restart; a simpler linear scan of #300.	O(n)	O(1)	contiguous run, reset on drop
688	Knight Probability in Chessboard	Return the probability that a knight remains on an `n×n` board after exactly `k` moves from a start cell, each move chosen uniformly among 8.	spread the probability at each cell to its 8 knight-move targets per step; off-board moves lose probability.	$O(k\cdot n^2)$	$O(n^2)$	8 knight moves, each prob 1/8
718	Maximum Length of Repeated Subarray	Find the length of the longest subarray common to two arrays (contiguous).	unlike LCS, the match must be contiguous, so a mismatch resets the running length to 0.	$O(m\cdot n)$	$O(m\cdot n)$	contiguous, no carry on mismatch
740	Delete and Earn	Deleting `nums[i]` earns its value but removes all copies of `nums[i]−1` and `nums[i]+1`; maximize earnings.	bucket points by value, then it becomes House Robber over the value line — you cannot take adjacent values.	O(n + max)	O(max)	bucket points by value
873	Length of Longest Fibonacci Subsequence	Given a strictly increasing array, find the length of the longest Fibonacci-like subsequence (each term = sum of the previous two).	a fib-like subseq is determined by its last two elements `(j, i)`; look up whether the required predecessor `arr[i]−arr[j]` exists earlier.	$O(n^2)$	$O(n^2)$	predecessor before arr[j]

#	Name	Description	Intuition	Time	Space	Key Implementation
887	Super Egg Drop	With `k` eggs and `n` floors, return the minimum number of moves to determine the critical floor in the worst case.	invert the problem — `dp[m][k]` = highest floor count solvable with `m` moves and `k` eggs; a move either breaks (test below) or survives (test above), plus the current floor.	O(k·log n)	O(n·k)	dp[m][k] = floors solvable in m moves
918	Maximum Sum Circular Subarray	Find the maximum subarray sum where the array is circular (subarray may wrap around).	the answer is either a normal max subarray (Kadane) or the total minus the minimum subarray (wrap); guard the all-negative case.	O(n)	O(1)	track min subarray for wrap
935	Knight Dialer	Count distinct phone numbers of length `n` dialable by knight moves on a phone keypad, modulo 1e9+7.	from each digit a knight can reach a fixed set of digits; spread counts across the adjacency map each step.	O(n)	O(1)	knight adjacency per digit
983	Minimum Cost For Tickets	Given travel days and the cost of 1-day, 7-day, and 30-day passes, return the minimum cost to cover all travel days.	on a travel day choose the cheapest pass covering it by looking back 1, 7, or 30 days; non-travel days inherit the prior cost.	O(lastDay)	O(lastDay)	non-travel day inherits cost
1027	Longest Arithmetic Subsequence	Find the length of the longest arithmetic subsequence (constant difference).	for each pair `(j, i)`, the arithmetic subseq ending at `i` with difference `d` extends the one ending at `j`.	$O(n^2)$	$O(n^2)$	per-index map keyed by difference
1048	Longest String Chain	Find the longest chain of words where each word is a predecessor of the next (insert exactly one char).	sort by length; for each word, try removing each character to find a shorter predecessor and extend its best chain.	$O(n \cdot L^2)$	O(n)	process shortest first
1137	N-th Tribonacci Number	Return the nth Tribonacci number (each term is the sum of the previous three).	the linear 1D recurrence with three rolling variables instead of two.	O(n)	O(1)	sum of previous three
1216	Valid Palindrome III	Return whether `s` becomes a palindrome after removing at most `k` characters.	the minimum removals to make `s` a palindrome equals `n − LPS(s)`; check whether that is `≤ k`.	$O(n^2)$	$O(n^2)$	interval DP for LPS
1218	Longest Arithmetic Subsequence of Given Difference	Find the longest arithmetic subsequence with a fixed difference `difference`.	with a known difference, the predecessor of `num` must be `num − difference`; one hashmap pass suffices.	O(n)	O(n)	fixed difference, value-keyed map
1269	Number of Ways to Stay in the Same Place After Some Steps	Count the ways to return to index 0 after `steps` moves (stay, left, or right) without leaving an array of size `arrLen`, modulo 1e9+7.	position can never exceed `steps/2`, so cap the reachable range; each step a position spreads to stay/left/right.	O(steps·min(arrLen, steps))	O(min(arrLen, steps))	cap reachable positions
1395	Count Number of Teams	Count triplets `(i, j, k)` with `i < j < k` whose ratings are strictly increasing or strictly decreasing.	fix the middle soldier `j`; the answer is its number of smaller-before × larger-after, plus the mirror case.	$O(n^2)$	O(1)	fix middle soldier
1567	Maximum Length of Subarray With Positive Product	Find the length of the longest subarray whose product is positive.	track the lengths of positive-product and negative-product runs ending at `i`; a negative number swaps them, a zero resets both.	O(n)	O(1)	zero resets both runs
1696	Jump Game VI	From index 0, each move jumps 1 to `k` indices forward; maximize the sum of values landed on, reaching the last index.	`dp[i]` = best score reaching `i` = `nums[i] +` max `dp[j]` in the window `[i−k, i−1]`; a monotonic queue keeps that window max in O(1).	O(n)	O(n)	monotonic queue of indices
1884	Egg Drop With 2 Eggs and N Floors	With exactly 2 eggs and `n` floors, return the minimum number of moves to determine the critical floor in the worst case.	`dp[i]` = min worst-case moves for `i` floors; dropping at floor `j` costs `1 + max(dp[j−1] from break, dp[i−j] from survive)`.	$O(n^2)$	O(n)	first drop at floor j

Trie `trie.md` (10 problems)

#	Name	Description	Intuition	Time	Space	Key Implementation
208	Implement Trie	Implement `insert(word)`, `search(word)`, and `startsWith(prefix)`.	each word is a path of letters down the tree; shared prefixes share the same path, so searching is just walking that path.	O(k) per operation, k = word length	O(n·k) total	Standard
211	Design Add and Search Words Data Structure	Same as Trie but `search` supports `.` which matches any single letter.		O(k) insert, O(26^k) search worst case (all `.`)	O(n·k)	when character is `.`, recursively try every non-null child.
212	Word Search II	Given a board and a list of words, find all words that exist in the board (connected adjacent cells, no reuse).		O(M·N·4·3^(L-1)) where L = max word length	O(total chars in words)	build a Trie from the word list; during grid DFS, follow the Trie path to prune branches that can't form any word. This avoids re-scanning the grid for each word independently.
648	Replace Words	Replace every word in a sentence with its shortest root from a dictionary. If multiple roots are prefixes, use the shortest.		O(n·k) build + O(sentence) search	O(dict·k)	insert dictionary into Trie. For each word in sentence, traverse the Trie and return immediately when `isEnd` is hit — that's the shortest matching root.
1268	Search Suggestions System	For each prefix of `searchWord` (length 1, 2, ..., n), return up to 3 products from `products` that share that prefix, in lexicographic order.		O(n·k·log n) build + O(k) search	O(n·k)	sort products first, then insert into Trie. At each node on the insertion path, cache the word (up to 3) — since products are sorted, the first 3 cached are always lexicographically smallest. Lookup is then O(1) per prefix.
745	Prefix and Suffix Search	Design a class `WordFilter` with `f(pref, suff)` that returns the index of the word with the given prefix and suffix. If multiple words match, return the largest index.		O(W·L²) build, O(p+s) query	O(W·L²) where W=words.length, L=max word length	precompute all prefix-suffix pairs
336	Palindrome Pairs	Given a list of unique words, find all pairs of distinct indices `(i, j)` such that `words[i] + words[j]` is a palindrome.	insert every word **reversed** into a Trie, tagging each end node with its index. For a query word, walk the Trie char by char; whenever the remainder of the word is itself a palindrome and we sit on a complete reversed word, those two concatenate into a palindrome.	O(n·k²) where n = words count, k = max word length	O(n·k)	words whose remaining suffix is a palindrome
676	Implement Magic Dictionary	Build a dictionary from a list of words. Implement `search(word)` that returns true if you can change **exactly one** character of `word` to make it match a dictionary word.	standard Trie insert; on search, DFS the Trie tracking how many characters have been changed so far. Allow exactly one mismatch.	O(k) insert, O(26^k) search worst case	O(n·k)	exactly one change required
720	Longest Word in Dictionary	Given a list of words, return the longest word that can be built one character at a time by other words in the list. If multiple, return the lexicographically smallest.	a word is buildable only if **every** prefix of it is also a complete word in the list. Insert all words, then DFS the Trie only through `isEnd` nodes — any reachable end node represents a fully buildable word.	O(n·k) build + O(n·k) DFS	O(n·k)	longest, ties broken lexicographically
1233	Remove Sub-Folders from the Filesystem	Given a list of folder paths, remove all sub-folders so only top-level folders remain. `"/a/b"` is a sub-folder of `"/a"`.	split each path into its `/`-separated components and build a Trie keyed by component (map-based, since components are arbitrary strings). Mark the node ending a folder. A folder is a sub-folder iff some ancestor node on its path is already an `isEnd` folder.	O(n·k) where n = folders, k = avg components	O(n·k)	map keyed by path component

Bit Manipulation `bit_manipulation.md` (17 problems)

#	Name	Description	Intuition	Time	Space	Key Implementation
136	Single Number	Array where every element appears twice except one. Find it.	XOR is its own inverse, so every duplicated pair cancels to 0 and only the lone element is left standing.	O(n)	O(1)	Standard
268	Missing Number	Array of n distinct numbers from [0, n] with one missing. Find it.	XOR-ing every index against every value pairs each present number with its index — the missing number's index has no partner to cancel.	O(n)	O(1)	XOR each index `i` with `nums[i]`. Indices 0..n XOR'd with the n elements — the missing index has no pair.
137	Single Number II	Every element appears three times except one. Find it.	XOR cancels pairs (mod 2); here we need a counter mod 3, so two state bits (`ones`, `twos`) cycle each bit through 1→2→0 and the survivor sits in `ones`.	O(n)	O(1)	Two-state XOR machine. `ones` tracks bits seen 1 mod 3 times, `twos` tracks bits seen 2 mod 3 times. When a bit reaches 3, it clears from both.
260	Single Number III	Two elements appear exactly once; all others appear twice. Find both.	XOR of all leaves `a ^ b`; any set bit in it is a bit where a and b differ, so it splits the array into two groups each holding one unique.	O(n)	O(1)	XOR all → get `a ^ b`. Use lowest set bit of `a ^ b` to partition nums into two groups (one containing `a`, one containing `b`). XOR each group independently.
191	Number of 1 Bits	Return the number of set bits in an unsigned integer.	Each `n & (n-1)` erases exactly one set bit, so the loop runs once per set bit — no need to scan all 32 positions.	O(k) k = number of set bits	O(1)	Standard
338	Counting Bits	Return an array `result` where `result[i]` = number of 1 bits in `i`, for i in [0, n].	`i` has the same set bits as `i >> 1` plus possibly its own last bit, so reuse the already-computed smaller answer instead of recounting.	O(n)	O(n)	DP relation — `i >> 1` is `i` with last bit removed (already computed), plus the last bit `i & 1`.
190	Reverse Bits	Reverse the 32 bits of an unsigned integer.	Peel the lowest bit off `n` and stack it onto `result` from the bottom — after 32 shifts the bit order is fully mirrored.	O(32) = O(1)	O(1)	Fixed 32 iterations — each time take the LSB of `n`, append it to `result`, then shift both.
201	Bitwise AND of Numbers Range	Return the AND of all numbers in `[left, right]`.	AND only keeps bits that are 1 in every number; the low bits flip somewhere in the range, so only the common high-bit prefix of `left` and `right` survives.	O(log n)	O(1)	right-shift both until equal to find shared prefix; shift back.
371	Sum of Two Integers	Return `a + b` without using `+` or `-`.	Addition splits into a carry-free sum (XOR) and the carries (AND shifted left); feed the carry back until nothing carries over.	O(32) = O(1)	O(1)	XOR computes bit sum without carry; AND shifted left computes carry. Repeat until no carry.
318	Maximum Product of Word Lengths	Given a list of words, find the maximum product `len(a) * len(b)` where `a` and `b` share no common letters.	Compress each word's letter set into a 26-bit integer so "share a letter?" becomes a single AND, replacing slow character-by-character comparison.	O(n² + n·k) k = avg word length	O(n)	encode each word as a 26-bit integer where bit `i` = 1 if letter `'a' + i` appears. Two words share a letter iff their bitmasks have any common bit (`mask[i] & mask[j] != 0`).
231	Power of Two	Given an integer `n`, return true if it is a power of two.	A power of two is a single 1 bit, and `n & (n-1)` clears that one bit to 0 — anything else leaves bits behind.	O(1)	O(1)	single-bit property of powers of 2
342	Power of Four	Given an integer `n`, return true if it is a power of four.	Powers of four are powers of two (one set bit) whose bit sits at an even position; masking against the odd-position mask `0xAAAAAAAA` must give 0.	O(1)	O(1)	Power of four must be power of two (one set bit) AND that bit must be at an even bit position (0, 2, 4, 6, ...). Mask `0xAAAAAAAA` has bits set at ALL odd positions; if `n & 0xAAAAAAAA == 0`, the set bit is at an even position.
287	Find the Duplicate Number	Given an array of `n+1` integers where each is in `[1, n]`, find the duplicate. No extra space, no modifying the array.	Following `i → nums[i]` turns the array into a linked list; a duplicate value means two indices point to the same node, forming a cycle whose entry is the duplicate.	O(n)	O(1)	Floyd's Cycle Detection. Treat the array as a linked list where `nums[i]` is the next node. Since there's a duplicate, two indices point to the same value → creates a cycle. Find cycle entry = duplicate.
405	Convert a Number to Hexadecimal	Given an integer `num`, return its hexadecimal representation as a lowercase string. For negative numbers use two's complement.	Hex is base 16, so each group of 4 bits maps to one hex digit; mask the lowest 4 bits with `num & 15` and shift right 4 at a time, using an unsigned shift so the two's-complement sign bit fills with zeros.	O(8) = O(1)	O(1)	Standard
461	Hamming Distance	Given two integers `x` and `y`, return the Hamming distance — the number of bit positions where they differ.	XOR sets exactly the bits where `x` and `y` differ, so the answer is just the number of set bits in `x ^ y`.	O(k) k = differing bits	O(1)	Standard
476	Number Complement	Given a positive integer `num`, return its complement: flip every bit up to and including the most significant set bit.	Build a mask of all 1s spanning the bit width of `num` (from the MSB down to bit 0), then XOR — flipping exactly the meaningful bits while leaving the leading zeros untouched.	O(log num)	O(1)	Standard

#	Name	Description	Intuition	Time	Space	Key Implementation
957	Prison Cells After N Days	8 prison cells (`cells[i]` is 0 or 1) update each day: a cell becomes 1 if both neighbors are equal, else 0. The two end cells always become 0 (they lack two neighbors). Return the state after `n` days.	With only 8 cells the state space is finite, so the configuration must cycle; detect the cycle length and reduce `n` modulo it to avoid simulating huge day counts.	O(min(n, cycle) · 8)	O(states)	Standard

Design `design.md` (24 problems)

#	Name	Description	Intuition	Time	Space	Key Implementation
146	LRU Cache	Design a data structure that follows the LRU (Least Recently Used) eviction policy. `get(key)` returns the value or -1. `put(key, value)` inserts/updates; evicts LRU when over capacity. Both operations must be O(1).	the HashMap answers "where is this key" instantly while the linked list keeps everything ordered by recency, so the victim to evict is always the node next to the tail.	O(1) get/put	O(capacity)	Standard
460	LFU Cache	Design a data structure for a Least Frequently Used cache. Both `get` and `put` must be O(1). On eviction, remove the least frequently used key; among ties in frequency, remove the least recently used.	bucket keys by how often they're used; the eviction victim is always the oldest key in the lowest-frequency bucket, which `minFreq` points straight at.	O(1) all operations	O(capacity)	frequency tracking on every access
380	Insert Delete GetRandom O(1)	Design a set where `insert`, `remove`, and `getRandom` all run in O(1) average time. `getRandom` returns any element with equal probability.	an array gives O(1) random indexing but O(n) middle-deletion; swapping the victim with the last element turns deletion into an O(1) pop while staying dense.	O(1) all ops	O(n)	swap target with last element
170	Two Sum III - Data structure design	Design a data structure with `add(number)` to store a number and `find(value)` returning true if any pair of stored numbers sums to `value`.	the complement check from classic Two Sum, but persisted across calls; the frequency map lets a single number satisfy a pair only when it appears at least twice.	O(1) add, O(n) find	O(n)	Standard
173	Binary Search Tree Iterator	Implement an iterator over a BST: `next()` returns the next smallest element and `hasNext()` reports whether one exists. Use O(h) memory.	an in-order traversal paused mid-flight — the stack stores exactly the unvisited ancestors, so memory stays proportional to tree height.	O(1) amortized next/hasNext	O(h)	Standard
244	Shortest Word Distance II	Initialize with a word list. `shortest(word1, word2)` returns the minimum index distance between the two words, supporting repeated queries efficiently.	because both index lists are sorted, the closest pair is found by always advancing the pointer at the smaller index — the same merge step as in sorted-list intersection.	O(n) init, O(a+b) per query	O(n)	Standard
245	Shortest Word Distance III	Given a word list and two words (which may be equal), return the shortest index distance. When `word1 == word2`, find the closest pair of distinct occurrences of that word.	one scan suffices for a single query; the equal-word case is just "closest pair of repeated occurrences," handled by remembering the previous match.	O(n)	O(1)	equal words use previous i
251	Flatten 2D Vector	Design an iterator over a 2D vector with `next()` and `hasNext()` that yields all elements in row-major order.	keep a `(row, col)` cursor and lazily skip empty rows; the skip logic concentrated in one helper keeps both methods correct.	O(1) amortized per call	O(1)	Standard
281	Zigzag Iterator	Given two lists, design an iterator that returns elements alternating between them; when one is exhausted, continue with the other.	a round-robin queue of iterators rotates through the lists; finished iterators simply fall out of the queue.	O(1) per call	O(k)	Standard
341	Flatten Nested List Iterator	Implement an iterator that flattens a nested list of integers, given the `NestedInteger` interface (`isInteger()`, `getInteger()`, `getList()`).	a stack performs the depth-first unwrap lazily — flattening happens only as far as the next requested integer.	O(1) amortized next, O(n) total	O(n)	Standard
346	Moving Average from Data Stream	Given a window size, design `next(val)` that returns the moving average of the last `size` values in the stream.	the running sum avoids re-summing the window; the queue only tracks which value leaves next.	O(1) per call	O(size)	Standard
348	Design Tic-Tac-Toe	Design an `n x n` Tic-Tac-Toe game. `move(row, col, player)` returns the winning player (1 or 2) if that move wins, else 0. Detect a winner in O(1) per move.	signing the contributions lets a single counter detect either player — only checking the four lines that the current move touches keeps it O(1).	O(1) per move	O(n)	Standard
381	Insert Delete GetRandom O(1) - Duplicates allowed	Like #380 but duplicates are allowed. `insert` returns true if the value was not already present, `remove` removes one occurrence, and `getRandom` picks proportionally to multiplicity.	the swap-with-last trick from #380 extends to duplicates by storing a set of indices per value instead of a single index. ← VARIATION: value → set of indices.	O(1) average all ops	O(n)	swap target with last
519	Random Flip Matrix	Design a structure over an `m x n` grid of zeros. `flip()` randomly picks a remaining 0-cell, sets it to 1, and returns its `[row, col]`. `reset()` restores all zeros.	virtual swap-to-end on a 1D index space gives uniform O(1) flips without materializing the full grid — the map only records cells that were moved.	O(1) flip, O(1) reset	O(flips between resets)	Standard
622	Design Circular Queue	Design a fixed-capacity circular queue with `enQueue`, `deQueue`, `Front`, `Rear`, `isEmpty`, `isFull`.	tracking `head` and `count` (rather than head/tail pointers) avoids the empty-vs-full ambiguity entirely.	O(1) all ops	O(k)	Standard
641	Design Circular Deque	Design a double-ended circular queue with `insertFront`, `insertLast`, `deleteFront`, `deleteLast`, `getFront`, `getRear`, `isEmpty`, `isFull`.	the #622 head/count model extends to both ends — inserting at the front just rotates `head` backward.	O(1) all ops	O(k)	rotate head backward
705	Design HashSet	Design a HashSet without built-in hash libraries, supporting `add`, `remove`, and `contains`.	separate chaining is the textbook collision strategy; a fixed prime-ish bucket count keeps chains short for typical inputs.	O(1) average per op	O(n)	Standard

#	Name	Description	Intuition	Time	Space	Key Implementation
706	Design HashMap	Design a HashMap without built-in hash libraries, supporting `put`, `get`, and `remove`.	identical structure to #705 but each node carries a value, so collisions are resolved by scanning a short chain of entries.	O(1) average per op	O(n)	update existing value
707	Design Linked List	Design a linked list supporting `get`, `addAtHead`, `addAtTail`, `addAtIndex`, and `deleteAtIndex`.	sentinels remove all null-edge cases for head/tail insertion, so every operation is the same splice on interior nodes.	O(index) get/add/delete	O(n)	Standard
729	My Calendar I	Implement `book(start, end)` for a half-open interval `[start, end)`; return true and add the booking if it does not overlap any existing one, else false.	the sorted map narrows the conflict check to at most two neighbors, giving O(log n) per booking.	O(log n) per book	O(n)	Standard
731	My Calendar II	Implement `book(start, end)`; return true unless adding it would cause a triple booking (three events overlapping at the same instant).	the difference array turns "max concurrent events" into a prefix-sum scan, and rolling back keeps the structure clean on rejection.	O(n) per book	O(n)	triple booking detected, roll back
732	My Calendar III	Implement `book(start, end)` returning the maximum `k` such that some instant is covered by `k` events (the largest k-booking so far).	the same difference-array sweep as #731, but here we report the peak concurrency rather than reject it.	O(n) per book	O(n)	track peak overlap
933	Number of Recent Calls	Implement `ping(t)` (calls arrive with strictly increasing `t`) returning how many pings occurred in the inclusive window `[t - 3000, t]`.	because times arrive sorted, expired pings are always at the front — a sliding window over a queue.	O(1) amortized per ping	O(window size)	Standard
1570	Dot Product of Two Sparse Vectors	Design a `SparseVector` initialized from an array; `dotProduct(other)` returns the dot product, taking advantage of sparsity.	since most entries are zero, storing and iterating only nonzeros makes the cost proportional to the number of nonzero terms, not the vector length.	O(n) init, O(min nonzeros) per dot product	O(nonzeros)	iterate fewer entries

Math `math.md` (61 problems)

#	Name	Description	Intuition	Time	Space	Key Implementation
7	Reverse Integer	Given a signed 32-bit integer `x`, return `x` with its digits reversed. Return 0 if the result overflows.	Peel the last digit with `%10` and push it onto a growing `result` with `*10`. The only hazard is overflow, so check capacity *before* the push.	O(log n)	O(1)	Overflow guard — before `result = result*10 + digit`, verify `result` is not past `Integer.MAX_VALUE/10` (or below `MIN_VALUE/10`).
9	Palindrome Number	Return true if integer `x` reads the same forwards and backwards. Negatives are never palindromes.	No need to reverse the whole number — build up the reversed second half until it meets or passes the shrinking first half. At the meeting point the two halves should match.	O(log n)	O(1)	Half reverse — loop while `x > reversed`; the middle digit (odd-length case) is dropped via `reversed / 10`.
50	Pow(x, n)	Implement `pow(x, n)`, computing `x` raised to the integer power `n`.	Squaring the base doubles the exponent reach, so each bit of `n` decides whether the current power of `x` joins the product — O(log n) multiplications instead of O(n).	O(log n)	O(1)	Handle negative exponent by inverting `x` and negating `N` (use `long` so `-Integer.MIN_VALUE` doesn't overflow); multiply into `result` only when the current exponent bit is set.
372	Super Pow	Compute `a^b mod 1337` where `b` is given as an array of digits (so `b` may be astronomically large).	Exponents add when powers multiply, and reading `b` digit by digit means `a^b = (a^(b/10))^10 * a^(b%10)`. Fold left across the digits, applying the modulus at every step to keep numbers small.	O(log n)	O(1)	Digit-by-digit modular exponentiation — at each new digit `d`, raise the running result to the 10th power and multiply by `a^d`, all mod 1337.
168	Excel Sheet Column Title	Given a column number, return its Excel-style title (1→"A", 27→"AA", 28→"AB").	It's base-26, but Excel has no zero digit — columns start at A=1, so the system is 1-indexed (a "bijective base 26"). Decrement before each digit to shift into the standard 0..25 range.	O(log n)	O(1)	1-indexed — do `columnNumber--` before extracting each digit so that `% 26` yields 0..25 mapping cleanly onto 'A'..'Z'.
171	Excel Sheet Column Number	Given an Excel column title (e.g. "AB"), return its column number (28).	Horner's rule for base-26, but each letter contributes `c-'A'+1` (A=1, not 0) because the system is 1-indexed.	O(L) L = title length	O(1)	1-indexed letters — `(c - 'A' + 1)` so 'A' maps to 1 rather than 0.
67	Add Binary	Given two binary strings `a` and `b`, return their sum as a binary string.	Grade-school addition from the rightmost digit, tracking a carry — identical to adding two linked-list numbers (#2), but the base is 2 and the operands are strings.	O(n) n = max length	O(n)	Char arithmetic per column — `(a.charAt(i)-'0') + (b.charAt(j)-'0') + carry`; emit `sum % 2` and carry `sum / 2`.
204	Count Primes	Count the number of prime numbers strictly less than `n`.	Instead of testing each number for primality, cross out every multiple of each prime once. When you reach an unmarked `i`, it's prime; its multiples below `i*i` were already crossed out by smaller primes, so start marking at `i*i`.	O(n log log n)	O(n)	Start marking at `i*i` — multiples `k*i` with `k < i` were already handled by the smaller factor `k`.
1201	Ugly Number III	Find the nth positive integer that is divisible by at least one of `a`, `b`, or `c`.	The count of valid numbers ≤ `x` is monotonic in `x`, so binary-search the smallest `x` whose count reaches `n`. Counting "≤ mid divisible by a, b, or c" is a classic inclusion-exclusion over the three sets, subtracting pairwise LCM overlaps and adding back the triple LCM.	O(log min(range))	O(1)	Inclusion-exclusion — `count = mid/a + mid/b + mid/c - mid/ab - mid/bc - mid/ac + mid/abc` using LCMs to avoid double counting.
29	Divide Two Integers	Divide two integers `dividend` and `divisor` without using multiplication, division, or mod, truncating toward zero. Clamp the result to the 32-bit signed range.	Repeated subtraction is too slow, so subtract the largest doubling of the divisor that still fits — this is long division in binary. Work in negatives so `Integer.MIN_VALUE` cannot overflow.	O(log n * log n)	O(1)	Exponential search of the divisor — double `divisor` and the matching quotient bit until it would exceed the remaining dividend.
59	Spiral Matrix II	Given `n`, generate an `n x n` matrix filled with the numbers `1` to `n*n` in spiral order.	Walk the four borders inward, shrinking the boundary after completing each side, filling a running counter.	O(n^2)	O(1) extra	Layer boundaries — maintain `top/bottom/left/right` and fill right, down, left, up in turn.
66	Plus One	Given a large integer represented as an array of digits (most significant first), increment it by one and return the resulting digit array.	Add one from the rightmost digit; a digit below 9 absorbs the carry and we're done, otherwise it becomes 0 and the carry propagates. If every digit was 9 we need one extra leading digit.	O(n)	O(1) extra (O(n) only for all-nines)	Early return — the moment a digit is incremented without rolling over, return; only an all-9 array needs a longer result.
118	Pascal's Triangle	Given `numRows`, generate the first `numRows` rows of Pascal's triangle.	Each interior entry is the sum of the two entries above it; the edges are always 1.	O(numRows^2)	O(numRows^2)	Row-by-row build — `row[j] = previous[j-1] + previous[j]`.
119	Pascal's Triangle II	Given a row index `rowIndex`, return that row of Pascal's triangle using only O(rowIndex) extra space.	Update a single row in place; iterating right-to-left lets each entry read its left neighbor before that neighbor is overwritten.	O(rowIndex^2)	O(rowIndex)	In-place reverse update — `row[j] += row[j-1]` scanning from the right.
149	Max Points on a Line	Given an array of points on a plane, return the maximum number of points lying on the same straight line.	Fix one point as the anchor; every other point defines a slope from it, and collinear points share that slope. Count the most common slope per anchor, using a	O(n^2)	O(n)	Slope histogram per anchor — key on `(dy/g, dx/g)` reduced by gcd, with sign normalization.

#	Name	Description	Intuition	Time	Space	Key Implementation
			reduced integer fraction as the key to avoid floating-point error.			
172	Factorial Trailing Zeroes	Given an integer `n`, return the number of trailing zeroes in `n!`.	A trailing zero comes from a factor of 10 = 2*5, and factors of 2 are far more plentiful than factors of 5, so just count the factors of 5. Multiples of 25 contribute an extra 5, of 125 another, and so on.	O(log n)	O(1)	Count factors of 5 — sum `n/5 + n/25 + n/125 + ...` via `n /= 5`.
202	Happy Number	A happy number reaches 1 by repeatedly replacing it with the sum of the squares of its digits; return whether `n` is happy.	The sequence either hits 1 or enters a cycle, so this is cycle detection — use Floyd's slow/fast pointers over the "sum of digit squares" transformation.	O(log n) per step	O(1)	Floyd cycle detection on a number transform — `slow = f(slow)`, `fast = f(f(fast))`.
223	Rectangle Area	Given the coordinates of two axis-aligned rectangles, return the total area they cover (counting the overlap once).	Total area is the sum of both areas minus the overlap; the overlap is itself a rectangle whose width and height are the intersections of the projections onto each axis (zero if they don't intersect).	O(1)	O(1)	Inclusion-exclusion of areas — overlap width = `min(rights) - max(lefts)` clamped at 0.
233	Number of Digit One	Given an integer `n`, count the total number of digit `1` appearing in all non-negative integers from 0 to `n`.	Count contributions of the digit 1 at each place value separately. For a given place, the count depends on the higher digits, the current digit, and the lower digits, which split cleanly into full cycles plus a partial cycle.	O(log n)	O(1)	Per-place digit counting — for each power-of-ten place, `count += high*place + adjust(cur, low, place)`.
243	Shortest Word Distance	Given an array of words and two distinct words, return the shortest distance (in indices) between any occurrence of the two words.	A single pass tracking the most recent index of each target word suffices; whenever both have been seen, the gap between the two latest positions is a candidate minimum.	O(n)	O(1)	Two latest-index trackers — update `result` whenever both indices are valid.
258	Add Digits	Repeatedly add all the digits of a non-negative integer until the result has a single digit; return it (the digital root).	The digital root has a closed form derived from numbers being congruent to their digit sum modulo 9: the answer is `0` for 0, otherwise `1 + (n−1) % 9`.	O(1)	O(1)	Digital root formula — no loop, just `1 + (num − 1) % 9`.
263	Ugly Number	An ugly number has no prime factors other than 2, 3, or 5; return whether `n` is ugly.	Divide out all factors of 2, 3, and 5; what remains is 1 exactly when no other prime factor exists. Non-positive numbers are never ugly.	O(log n)	O(1)	Factor stripping — repeatedly divide by each of 2, 3, 5 and check the remainder is 1.
292	Nim Game	You and an opponent alternate removing 1 to 3 stones; whoever takes the last stone wins. Given `n` stones and you move first, return whether you can win.	If `n` is a multiple of 4 you always lose, because whatever you take (1-3) the opponent can complete a group of four, keeping `n` a multiple of 4 on your turn until 0.	O(1)	O(1)	Multiple-of-four loss — win exactly when `n % 4 != 0`.
319	Bulb Switcher	There are `n` bulbs initially off; in round `i` you toggle every `i`-th bulb. Return how many bulbs are on after `n` rounds.	Bulb `k` is toggled once per divisor of `k`, ending on only if it has an odd number of divisors — which happens exactly for perfect squares. The count of perfect squares up to `n` is `floor(sqrt(n))`.	O(1)	O(1)	Perfect-square count — answer is `(int) sqrt(n)`.
326	Power of Three	Given an integer `n`, return whether it is a power of three.	Within the 32-bit range the largest power of three is `3^19 = 1162261467`; any power of three must divide it exactly, so a single divisibility test works.	O(1)	O(1)	Max-power divisibility — `n > 0 && 1162261467 % n == 0`.
357	Count Numbers with Unique Digits	Given `n`, count all numbers `x` with `0 <= x < 10^n` that have no repeated digits.	Count by length: the first digit has 9 choices (1-9), the next 9 (any but the first), then 8, 7, ..., multiplying as digits are added. Sum these counts across lengths 1 to n, plus the single number 0.	O(n)	O(1)	Permutation counting per length — `9 * 9 * 8 * ...` accumulated.
365	Water and Jug Problem	Given two jugs of capacities `x` and `y` and a target `target`, determine if you can measure exactly `target` liters using fill/empty/pour operations.	Any amount measurable is a non-negative integer combination of `x` and `y`, which by Bezout's identity is exactly the multiples of `gcd(x, y)`. So the target must be reachable in total capacity and divisible by the gcd.	O(log(min(x,y)))	O(1)	Bezout / gcd test — `target % gcd(x, y) == 0` with `target <= x + y`.
367	Valid Perfect Square	Given a positive integer `num`, return whether it is a perfect square, without using any built-in sqrt function.	The square root lies in `[1, num]` and squaring is monotonic, so binary search the candidate root and compare its square (in `long` to avoid overflow) against `num`.	O(log n)	O(1)	Binary search on the root — midpoint `k = i + (j−i)/2`, compare `k*k` to `num`.
397	Integer Replacement	Given a positive integer `n`, each step you may replace it with `n/2` if even, or `n+1` / `n−1` if odd; return the minimum steps to reach 1.	Halving is always best when possible. For odd `n`, choosing `+1` vs `−1` should expose more trailing zeros to halve next; `n == 3` is the special case where `−1` is better.	O(log n)	O(1)	Greedy on the last two bits — for odd `n != 3`, add 1 when `(n & 2) != 0` else subtract 1.
398	Random Pick Index	Given an array `nums`, implement `pick(target)` returning a uniformly random index where `nums[index] == target`.	Reservoir sampling lets us pick uniformly in one pass without storing all matching indices: the `k`-th matching element replaces the current pick with probability `1/k`.	O(n) per pick	O(1)	Reservoir sampling of size 1 — keep the index with probability `1/count` as matches stream by.
400	Nth Digit	Given `n`, return the `n`-th digit in the infinite sequence of concatenated positive integers `123456789101112...`.	Numbers group by digit-length: there are `9*10^(d−1)` numbers with `d` digits, contributing `d * 9 * 10^(d−1)` digits. Subtract whole groups to find the length, locate the exact number, then index into it.	O(log n)	O(1)	Length-bucket walk — subtract `count * digitLength` until `n` falls inside the current bucket.
412	Fizz Buzz	Return a list of strings for `1..n` where multiples of 3 become "Fizz", multiples of 5 become "Buzz", multiples of both	Build each entry by concatenating "Fizz" and/or "Buzz" based on divisibility; if neither applies, use the number's string.	O(n)	O(1) extra	Concatenate divisibility tags — append "Fizz" on `%3`, "Buzz" on `%5`.

#	Name	Description	Intuition	Time	Space	Key Implementation
		become "FizzBuzz", and others the number itself.				
441	Arranging Coins	You have `n` coins to form a staircase where the `k`-th row has exactly `k` coins; return the number of complete rows.	After `k` complete rows you've used `k(k+1)/2` coins, so the answer is the largest `k` with `k(k+1)/2 <= n`. Binary search this `k` (or solve the quadratic).	O(log n)	O(1)	Binary search on rows — compare triangular number `k(k+1)/2` against `n` using `long`.
453	Minimum Moves to Equal Array Elements	In one move you increment `n-1` elements of the array by 1; return the minimum moves to make all elements equal.	Incrementing all but one is equivalent (for the purpose of equalizing) to decrementing a single element by 1. So the answer is the total amount each element exceeds the minimum: `sum - n * min`.	O(n)	O(1)	Relative-decrement reframing — total moves = `sum(nums) - n * min(nums)`.
458	Poor Pigs	With `buckets` buckets one of which is poisoned, `minutesToDie` for poison to act, and `minutesToTest` total time, return the minimum pigs needed to find the poisoned bucket.	Each pig can be tested `t = minutesToTest / minutesToDie` times, so it has `t + 1` distinguishable states (died after round 1..t, or survived). With `p` pigs we cover `(t+1)^p` buckets, so we need the smallest `p` with `(t+1)^p >= buckets`.	O(log buckets)	O(1)	Base-(tests+1) counting — `p = ceil(log_{t+1}(buckets))`.
470	Implement Rand10() Using Rand7()	Given `rand7()` uniform in [1,7], implement `rand10()` uniform in [1,10].	Two calls form a uniform value in [1,49]; reject anything above 40 and map the remaining 40 outcomes evenly onto [1,10]. Rejection keeps the distribution exactly uniform.	O(1) expected	O(1)	Rejection sampling on a 7x7 grid — accept index `< 40`, return `index % 10 + 1`.
492	Construct the Rectangle	Given a target `area`, return the dimensions `[L, W]` with `L >= W`, `L * W == area`, and `L - W` minimized.	The most square-like factor pair minimizes the difference, so start `W` at `floor(sqrt(area))` and walk down until it divides `area`.	O(sqrt(area))	O(1)	Search down from sqrt — first divisor `W <= sqrt(area)` gives the closest pair.
495	Teemo Attacking	Given sorted attack `timeSeries` and a poison `duration`, return the total time the target is poisoned (overlaps counted once).	Each attack would add `duration`, but if the next attack comes before the current poison ends, only the gap between attacks counts. Sum the capped gaps and add a full duration for the last attack.	O(n)	O(1)	Capped gaps — add `min(gap, duration)` between consecutive attacks.
528	Random Pick with Weight	Given an array `w` of weights, implement `pickIndex()` returning index `i` with probability proportional to `w[i]`.	Build prefix sums so the cumulative weights partition `[1, total]` into intervals; draw a random target in that range and binary-search the first prefix sum reaching it.	O(log n) per pick, O(n) build	O(n)	Prefix-sum + binary search — find leftmost prefix `>= target`.
593	Valid Square	Given four points, return whether they form a valid square (positive area).	Among the six pairwise squared distances of a square there are exactly two distinct values: four equal sides and two equal (larger) diagonals, with the diagonal twice the side. Use squared distances to stay in integers.	O(1)	O(1)	Two-distinct-distance check — collect six squared distances; require exactly two values, the smaller nonzero, the larger double it.
598	Range Addition II	Given an `m x n` matrix of zeros and operations each incrementing the top-left `a x b` submatrix, return how many cells hold the maximum value.	Every operation always covers cell (0,0), so the maximum value sits in the intersection of all operation rectangles — its area is the product of the smallest row bound and smallest column bound.	O(ops)	O(1)	Intersection area of all ops — `min(all a) * min(all b)`.
633	Sum of Square Numbers	Given a non-negative integer `c`, decide whether there exist non-negative integers `a, b` with `a*a + b*b == c`.	Squeeze two pointers from `0` and `floor(sqrt(c))`: if the squared sum is too small move the low pointer up, too large move the high pointer down.	O(sqrt(c))	O(1)	Two pointers on squares — `a` from 0 up, `b` from sqrt(c) down.
656	Coin Path	Given `coins` (cost per index, `-1` blocked) and a max jump `maxJump`, find the lexicographically smallest cheapest path of indices from index 0 to the last.	Work backwards with DP: the best cost from index `i` is its cost plus the cheapest reachable next index within `maxJump`. Filling from the right and preferring the smaller next index yields the lexicographically smallest path on ties.	O(n * maxJump)	O(n)	Backward DP with path reconstruction — `cost[i] = coins[i] + min over j in (i, i+maxJump]`, tie-break to smaller `j`.
781	Rabbits in Forest	Each surveyed rabbit reports how many other rabbits share its color; given the `answers`, return the minimum number of rabbits in the forest.	Rabbits answering `k` form groups of size `k+1`. For each answer value, every full group of `k+1` such replies fills one color group; partial groups still require a whole `k+1` block.	O(n)	O(n)	Group-rounding by answer — for count `c` of answer `k`, add `ceil(c/(k+1)) * (k+1)`.
789	Escape The Ghosts	Starting at the origin with a `target`, and given `ghosts` positions, you all move simultaneously in Manhattan steps; return whether you can reach the target before any ghost catches you.	You win iff you reach the target strictly before every ghost. Since all move optimally with Manhattan distance, compare your distance from origin to each ghost's distance to the target; if no ghost is at least as close, you escape.	O(g)	O(1)	Manhattan-distance race — you win when your distance to target < every ghost's distance to target.
792	Number of Matching Subsequences	Given a string `s` and an array of `words`, return how many words are subsequences of `s`.	Rather than scan `s` per word, bucket words by their next-needed character. Sweep `s` once; each character releases its waiting words, advancing them to their next character bucket, and any word that runs out of characters is a match.	O(|s| + total word length)	O(total word length)	Waiting lists per next-char — advance words bucketed by the character they currently need as `s` streams.
829	Consecutive Numbers Sum	Given `n`, return the number of ways to write it as a sum of consecutive positive integers.	A run of `k` consecutive integers starting at `a` sums to `k*a + k(k-1)/2`, so `n - k(k-1)/2` must be positive and divisible by `k`. Try each `k` while the triangular offset stays below `n`.	O(sqrt(n))	O(1)	Count valid run-lengths — for each `k`, valid iff `(n - k(k-1)/2) % k == 0` and positive.

#	Name	Description	Intuition	Time	Space	Key Implementation
836	Rectangle Overlap	Given two axis-aligned rectangles `rec1` and `rec2` as `[x1,y1,x2,y2]`, return whether they overlap in positive area.	Two rectangles overlap iff their projections on both axes overlap; equivalently, neither is entirely to one side of the other on either axis.	O(1)	O(1)	Separating-axis on both projections — overlap iff `x` ranges and `y` ranges each intersect.
858	Mirror Reflection	A square room with mirrored walls has receptors at corners 0,1,2; a laser leaves the southwest corner and first travels to the east wall at height `q` (room side `p`). Return which receptor it first meets.	Unfold the reflections so the beam travels straight; it hits a corner after going up a multiple of `p` that is also a multiple of `q`, i.e. `lcm(p,q)`. The parities of how many room-widths and room-heights that represents determine the receptor.	O(log(min(p,q)))	O(1)	Parity of `lcm/p` and `lcm/q` — reduce `p,q` by gcd, then the receptor follows from which is odd/even.
869	Reordered Power of 2	Given an integer `n`, return whether some permutation of its digits (no leading zero) equals a power of 2.	Two numbers are digit-permutations iff they have the same multiset of digits. So compute a digit-count signature of `n` and compare it against the signatures of all powers of 2 within the 32-bit range.	O(log^2 n)	O(1)	Digit-count signature match — compare sorted-digit fingerprint against each power of 2.
939	Minimum Area Rectangle	Given points in the plane, find the minimum area of an axis-aligned rectangle formed by four of them, or 0 if none exists.	A rectangle is fixed by its diagonal corners; for each pair of points with different x and y, the other two corners are determined, so check whether both exist in a point set.	O(n^2)	O(n)	Diagonal-pair lookup — for each pair, test if the complementary corners are present in a hash set.
963	Minimum Area Rectangle II	Given points in the plane, find the minimum area of any rectangle (any orientation) formed by four of them, or 0.	A rectangle's diagonals share the same midpoint and the same length. Group point pairs by `(midpoint, diagonal length)`; any two pairs in a group are the two diagonals of a rectangle, whose sides come from the corner vectors.	O(n^2) groups, worst O(n^2) per group	O(n^2)	Group diagonals by midpoint+length — within a group, combine pairs and compute area via vectors.
1041	Robot Bounded In Circle	A robot starts at the origin facing north and repeats `instructions` ('G' forward, 'L'/'R' turn) forever; return whether it stays within some bounded circle.	After one pass, the robot is bounded iff it returns to the origin, or it no longer faces north — a non-north heading guarantees the net displacement rotates and cancels over at most four cycles.	O(n)	O(1)	One-cycle check — bounded iff back at origin OR final direction != initial north.
1160	Find Words That Can Be Formed by Characters	Given `words` and a string `chars`, return the total length of words that can be formed using letters of `chars` (each letter used at most as often as it appears).	Count available letters once; a word is formable iff its own letter counts never exceed the available counts. Sum lengths of formable words.	O(total characters)	O(1)	Frequency containment — compare per-word letter counts against the `chars` budget.
1304	Find N Unique Integers Sum up to Zero	Given `n`, return any array of `n` unique integers that sum to zero.	Pair each positive `i` with its negation `-i`; that cancels to zero, and add a lone 0 if `n` is odd.	O(n)	O(1) extra	Symmetric pairs — emit `i` and `-i`, plus a central 0 for odd `n`.
1344	Angle Between Hands of a Clock	Given an `hour` and `minutes`, return the smaller angle (in degrees) between the hour and minute hands.	The minute hand moves 6 degrees per minute; the hour hand moves 30 degrees per hour plus 0.5 degree per minute. The answer is the absolute difference, folded to at most 180.	O(1)	O(1)	Per-hand angular position — minute = `6*m`, hour = `30*(h%12) + 0.5*m`, take `min(diff, 360-diff)`.
1583	Count Unhappy Friends	Given `n` friends, their `preferences`, and a `pairs` assignment, count friends who are unhappy — paired with someone they prefer less than another friend who also prefers them over their own partner.	Precompute each friend's preference rank for every other friend. A friend `x` (paired with `y`) is unhappy if some `u` ranks higher than `y` for `x`, and `u` ranks `x` higher than `u`'s own partner. Compare ranks pairwise.	O(n^2)	O(n^2)	Rank-matrix mutual-preference scan — `x` unhappy if exists `u` with `rank[x][u] < rank[x][y]` and `rank[u][x] < rank[u][partner[u]]`.
1588	Sum of All Odd Length Subarrays	Given an array `arr`, return the sum of all elements over all odd-length contiguous subarrays.	Instead of enumerating subarrays, count how many odd-length subarrays include each index. With `i+1` choices for the left boundary and `n-i` for the right, the number of odd-length ones is computed in closed form, weighting each element.	O(n)	O(1)	Per-element contribution — element `i` appears in `ceil(((i+1)*(n-i))/2)` odd-length subarrays.
1646	Get Maximum in Generated Array	Build array `nums` where `nums[0]=0`, `nums[1]=1`, `nums[2i]=nums[i]`, `nums[2i+1]=nums[i]+nums[i+1]`; return its maximum for size `n`.	Generate the array directly from the recurrence, tracking the running maximum. Handle the tiny base cases for `n < 2`.	O(n)	O(n)	Direct recurrence fill — even index copies, odd index sums the two halves.
1823	Find the Winner of the Circular Game	`n` friends in a circle eliminate every `k`-th friend repeatedly; return the 1-indexed winner (Josephus problem).	The Josephus recurrence builds the survivor's position from a circle of size 1 upward: the winner in a circle of `i` is `(winner_{i-1} + k) % i`. Convert the final 0-indexed answer to 1-indexed.	O(n)	O(1)	Josephus recurrence — `winner = (winner + k) % i` for `i = 2..n`.
1969	Minimum Non-Zero Product of the Array Elements	For the array of all integers `1..2^p - 1`, you may swap bits between elements any number of times; return the minimum non-zero product modulo `1e9+7`.	Pair the largest value `2^p - 1` (kept whole) with the rest: each other pair can be made `(2^p - 2)` and `1`, so the product is `(2^p - 1) * (2^p - 2)^(2^(p-1) - 1)` — compute with modular fast	O(p)	O(1)	Pairing + modular fast power — `(max) * pow(max-1, half-1) mod M`.

#	Name	Description	Intuition	Time	Space	Key Implementation
			power, but the base of the exponent must use the true (non-reduced) count.			